

RFC: Password-Based Key Management for the Encryption VFD

Clay Carper clay.carper@lifeboat.llc

In the context of the Encryption VFD, key management is an essential component of providing a comprehensive data security solution. This document describes the proposed password-based key management architecture used to create, protect, recover, and manage Data Encryption Keys (DEKs). As an initial implementation, a local, operating system-agnostic key management mechanism is proposed in which a randomly generated DEK is encrypted, or wrapped, using a password-derived Key Encryption Key (KEK). At runtime, the user's password is used to reconstruct the KEK, authenticate the DEK+ object, recover the DEK, and store it within a `libgcrypt` secure memory pool for use by the Encryption VFD. Although the architecture is designed to accommodate additional authentication mechanisms in the future, the initial implementation supports only password-derived KEKs. Definitions of the terminology used throughout this document are provided in Appendix A – Glossary.

Table of Contents

1	Introduction	4
2	Key Management From a User Perspective	4
2.1.1	Selecting a Key Management Workflow	5
2.1.2	Creating an Encrypted HDF5 File with Terminal-Based Input	6
2.1.3	Creating an Encrypted HDF5 File Without Terminal-Based Input	7
2.1.4	Opening an Encrypted HDF5 File with Terminal-Based Input	8
2.1.5	Opening an Encrypted HDF5 File Without Terminal-Based Input	9
2.1.6	Password Management Overview.....	10
2.1.7	Change password with Terminal-Based Input.....	10
2.1.8	Change password without Terminal-Based Input	11
2.1.9	Add a password with Terminal-Based Input.....	12
2.1.10	Add a password without Terminal-Based Input	13
2.1.11	Remove a password with Terminal-Based Input.....	14
2.1.12	Remove a password without Terminal-Based Input	14
2.1.13	Migrating Between Supported DEK Handling Mechanisms	15
3	Implementation Outline	16
3.1	High-Level Usage Description	16
3.2	DEK+ Format Summary	18
3.3	Password Usage.....	18
3.4	User-Interaction and Policy Model	19
3.5	Usage with the Encryption VFD	20
3.6	H5keytools Companion Utility	20
3.6.1	Purpose	20
3.6.2	Supported Operations	21
3.6.2.1	Create DEK	21
3.6.2.2	Create DEK+ Object.....	21
3.6.2.3	Validate DEK+ Object	21
3.6.2.4	Change Password	21
3.6.2.5	Remove Password	22
3.6.2.6	Add Password.....	22

3.6.2.7	Update KDF parameters.....	22
3.6.2.8	Future Format Migration	22
3.6.3	Transferring Between Supported DEK Handling Mechanisms	23
3.6.3.1	Convert from Secure to Secure	23
3.6.3.2	Convert from Insecure to Secure	23
3.6.3.3	Convert from Secure to Insecure	24
3.6.4	Build-Time Protection.....	24
4	Theory of Operations	25
4.1	Create a New DEK+ Object	25
4.2	Interacting with an Existing Encrypted File.....	25
4.3	Rotating the KEK using h5keytools	26
5	Technical Details	26
5.1	Encryption VFD Overview	27
5.2	DEK+ Format	27
5.2.1	DEK+ Object Layout	28
5.2.2	DEK+ Header Structure.....	30
5.2.3	DEK+ Header Flags.....	34
5.2.4	Serialization and Endianness	35
5.2.5	Cryptographic Construction	35
5.2.6	Secret and Non-Secret Material	37
5.3	Code Components	37
5.3.1	Runtime Policy and Operation Flags	38
5.3.2	Public Control Operations (H5FDctl)	40
5.3.3	Internal High-Level DEK+ Operations	41
5.3.4	Omnibus Input and Output Structures.....	42
5.3.5	Internal DEK+ Read/Write Helpers.....	50
5.3.6	Internal Cryptographic Helpers	51
5.3.7	Password and Policy Helpers.....	53
5.4	Call-Stack Outlines	54
5.4.1	Creating a New DEK+ Object	54
5.4.2	Unlock Existing DEK+ Object.....	54

5.5 Rekey Workflow (h5keytools) (Password Change/Removal)55

5.6 Testing Plan55

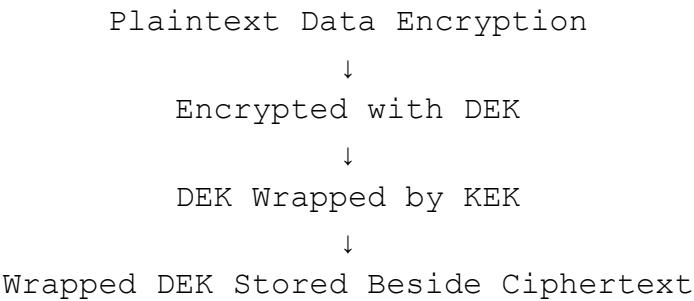
Appendix A – Glossary56

1 Introduction

In the context of the Encryption VFD, key management is an essential component for providing users with a comprehensive data security solution. As a first step, a local, operating system-agnostic key-protection mechanism will be utilized. Under this model, a random Data Encryption Key (DEK) is generated locally and then encrypted, or wrapped, using a password-derived Key Encryption Key (KEK). At runtime¹, the user’s password is then used to recover the DEK, which is then stored in a `libgcrypt` secure memory pool and used by the Encryption VFD. While this solution provides a framework appropriate for utilizing other authentication methods, at the time of writing only password-derived KEKs will be supported. Concerning the terminology used throughout this document, definitions are made available in Appendix A – Glossary.

2 Key Management From a User Perspective

In order to properly utilize the proposed password-based key management solution, it is crucial to understand the underlying principle that inspired this approach, envelope encryption. Envelope encryption refers to the process of wrapping an encryption key using a different, secondary encryption key. Simply stated, the two keys used are referred to as a Data Encryption Key (DEK), which is used to encrypt data (i.e., the key used to secure the content of an encrypted HDF5 file) and a Key Encryption Key (KEK), which is used as the encryption key used to wrap a DEK. In order to create a clean nomenclature, a DEK that is encrypted using a KEK is often referred to as a wrapped DEK. Generally speaking, the underlying encryption model is as follows:



The KEK is never stored directly. Instead, it is reconstructed whenever needed from the user’s password and the information stored in the DEK+ format. The recovered KEK is then used to unwrap

¹ In environments where an interactive terminal is unavailable, users may create an empty-string password, allowing the wrapped DEK to be recovered without prompting for a password. This mode is intended primarily for automated testing, batch processing, or HPC environments where interactive authentication is impractical.

the DEK, after which the KEK is discarded while the DEK remains available for use by the Encryption VFD. What this means in practice is the password-protected DEK is typically never directly exposed to the user, reducing the amount of knowledge required to work with an encrypted HDF5 file. Further details relating to the creation and usage of these principles with respect to an encrypted HDF5 file are discussed later in this section.

The information required to recover the DEK is stored within a binary object referred to throughout this document as the DEK+ format. Rather than storing the cleartext DEK (unless configured otherwise) or the user's password, the DEK+ format stores the wrapped DEK together with the cryptographic metadata required to reconstruct the KEK, verify the integrity of the stored information, and recover the DEK during future operations. For the initial implementation, the DEK+ format is expected to reside within a reserved page of the encrypted HDF5 file, allowing the Encryption VFD to automatically locate the information needed to recover the DEK whenever the file is opened. Because the DEK+ format contains only encrypted key material and supporting metadata, users are not expected to directly inspect or modify its contents during normal operation.

Envelope encryption also provides a natural mechanism for accommodating password changes. In this context, a password change will result in the creation of a new KEK. This process is commonly referred to as a key rotation operation. Under the principles of envelope encryption, the underlying DEK remains constant, allowing for the periodic rotation of KEKs. When a KEK is rotated, the underlying data need not be re-encrypted [2]. Instead, the associated DEK is simply unwrapped with the old KEK, then rewrapped with the new KEK.

2.1.1 Selecting a Key Management Workflow

Users select the desired key management behavior through the Encryption VFD configuration, the h5keytools companion utility, or the corresponding public API. Users are not expected to construct internal operation flags or modify the flags stored within the DEK+ object. These values are selected and maintained automatically by the implementation based on the requested operation.

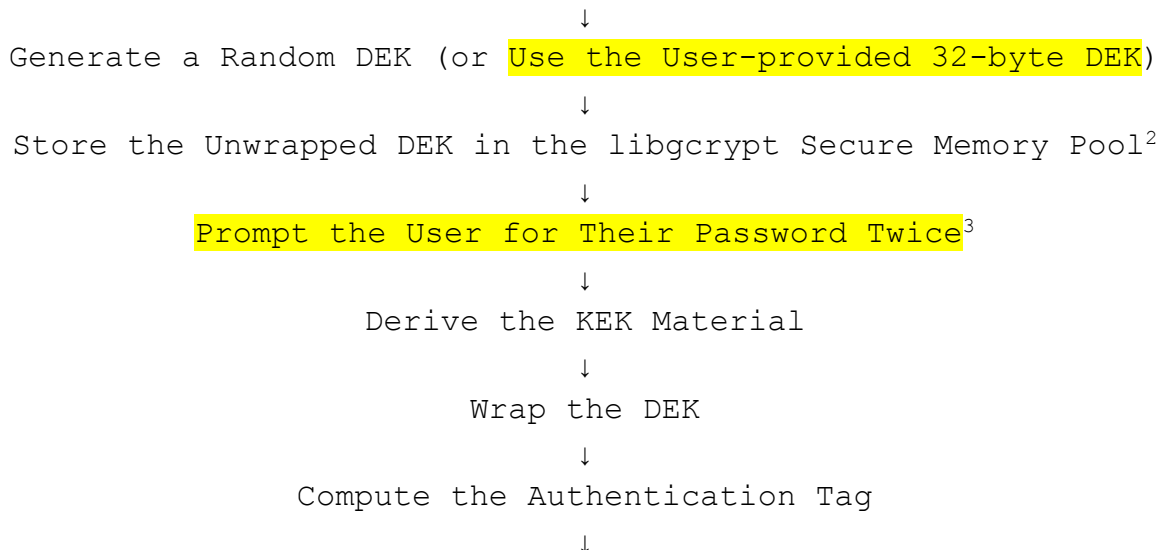
The capabilities available to the user may depend on how the Encryption VFD and h5keytools utility were built. Security-sensitive functionality, such as empty-string-password support, cleartext DEK storage, cleartext DEK export, or creation of an external DEK+ copy, may require the corresponding build option to be enabled. If the user requests an operation that is not available in the installed build, the operation will fail with an error identifying the unavailable capability. For interactive workflows, h5keytools or the Encryption VFD collects the required password and confirmation input through the terminal. For non-interactive workflows, the calling application supplies the required values programmatically. Both workflows invoke the same underlying implementation and produce equivalent cryptographic results when supplied with the same inputs and policies.

The use cases discussed in the remainder of this section are intended to reflect how the user is expected to interact with the password-based key management solution presented in this document. For each operation outlined in this section, it is assumed that the user has determined whether they will use an interactive or non-interactive workflow and have supplied the appropriate build option to be enabled.

2.1.2 Creating an Encrypted HDF5 File with Terminal-Based Input

- Creating a new encrypted HDF5 file with DEK+ support will automatically generate a new DEK+ object and store the recovered plaintext DEK within the libgcrypt secure memory pool for immediate use by the Encryption VFD. Unless the user explicitly supplies an existing 32-byte DEK, a new cryptographically secure random DEK is generated.
- Although a user may optionally supply an existing 32-byte DEK through the Encryption VFD configuration string, doing so is generally discouraged because reuse of the same DEK across multiple encrypted HDF5 files increases the impact of a compromised DEK.
- The workflow assumes that interactive terminal input is available. Consequently, the user is prompted to enter their password during file creation, allowing the Encryption VFD to derive the KEK without exposing the password through the configuration string.
- The procedure below illustrates the general sequence of operations performed when creating a new encrypted HDF5 file using interactive password entry. Highlighted components indicate where user interaction is expected to occur.

User Calls H5Fcreate() to Create an Encrypted HDF5 File



² The unwrapped DEK is retained in the libgcrypt secure memory pool for the life of the encrypted HDF5 file. Should the HDF5 file be closed or the library experience a crash, the unwrapped DEK will be securely erased.

³ The User is prompted for a password in two subsequent requests. If the provided input differs between the two entries, the user will be told the password does not match and will be asked to set a password again. If a User does not wish to associate a non-trivial password with the KEK, they may simply press **Enter** when prompted. This results in an empty password (``) being used during KEK derivation and allows for the password prompt to be bypassed when permitted by the Encryption VFD configuration. This mode should only be used when interactive password entry is undesirable and a cleartext DEK is not desired. Note that using an empty password is functionally equivalent to removing the password-based protection provided by this design, as it becomes trivial to recover the wrapped DEK.

Write the DEK+ Format into the Second Page of the Encrypted HDF5 File

↓

Encrypt or Decrypt Data in the HDF5 File

Once the encrypted HDF5 file has been created, the recovered DEK remains resident within the libgcrypt secure memory pool for the lifetime of the open file. Consequently, the user is prompted for their password only during file creation and is not required to repeatedly authenticate as additional encrypted file content is accessed. The password-derived KEK exists only long enough to protect the DEK and is securely erased immediately afterward.

2.1.3 Creating an Encrypted HDF5 File Without Terminal-Based Input

- This workflow performs the same DEK generation and DEK+ creation process described in Section 2.1.1, except the password is supplied programmatically rather than through an interactive terminal prompt.
- As with the interactive workflow, the user may optionally provide an existing 32-byte DEK instead of allowing the Encryption VFD to generate a random DEK. Unless explicitly required by the surrounding application, generation of a new random DEK remains the recommended approach.
- The user may use a trivial password by passing "" rather than passing a non-trivial password (i.e., "mykepasswordgoeshere"). Because the password is supplied directly through the configuration string, it exists in ordinary application memory before KEK derivation. Consequently, this workflow should be reserved for trusted systems or environments where interaction authentication is unavailable.
- The procedure below illustrates the general sequence of operations performed during non-interactive DEK+ creation. Highlighted components indicate where application-provided input is consumed.

User Calls H5Fcreate() to Create an Encrypted HDF5 File

↓

Generate a Random DEK (or Use the User-provided 32-byte DEK)

↓

Store the Unwrapped DEK in a libgcrypt Secure Memory Pool

↓

Load the Password from the Configuration String

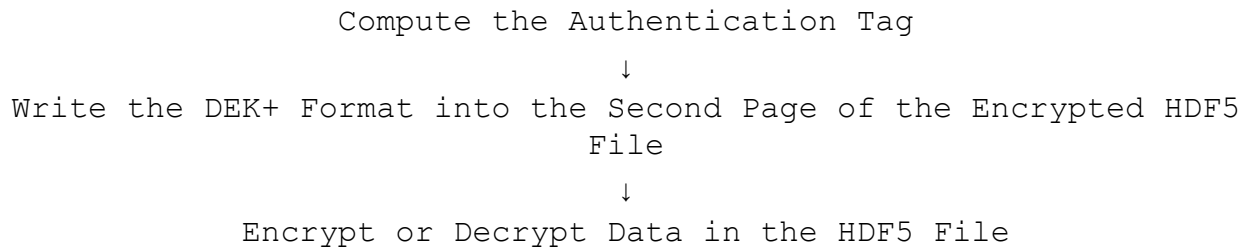
↓

Derive the KEK Material

↓

Wrap the DEK

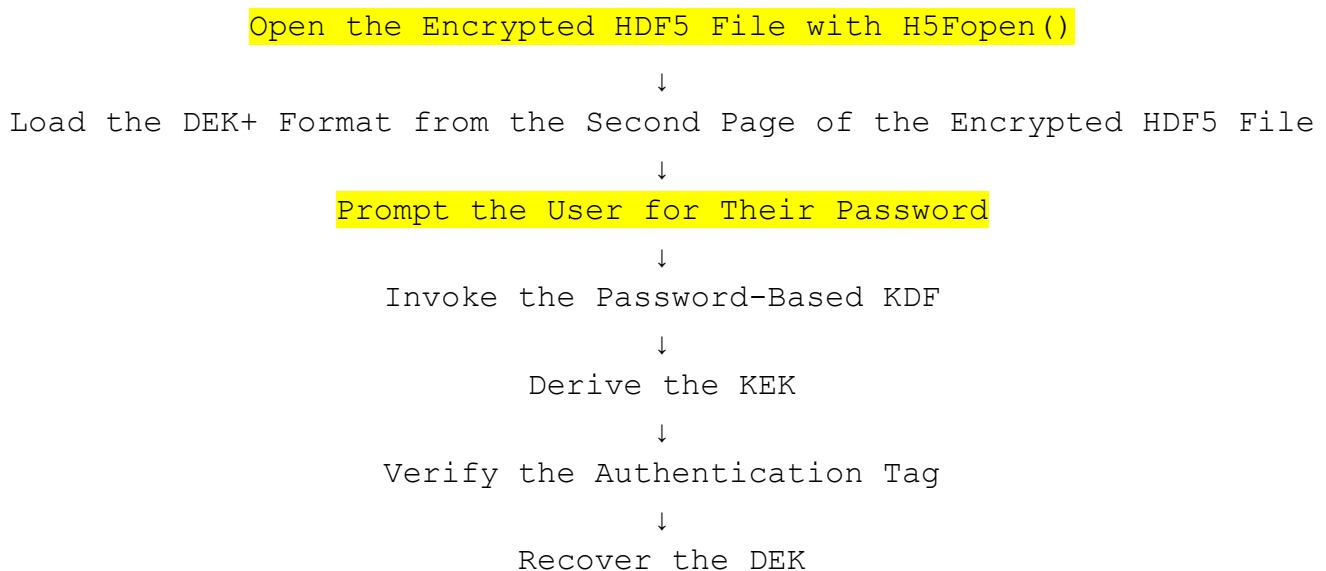
↓



Although the password is supplied programmatically rather than entered through the terminal, the resulting cryptographic protection is identical to the interactive workflow. The only practical difference is the mechanism by which the password is provided to the Encryption VFD. Because the password exists in cleartext within application memory before key derivation, the workflow should be reserved for trusted systems or environments where interactive input is unavailable.

2.1.4 Opening an Encrypted HDF5 File with Terminal-Based Input

- Opening an encrypted HDF5 file recovers the existing DEK stored within the DEK+ format rather than generating a new encryption key. Once recovered, the DEK remains resident within the `libgcrypt` secure memory pool for the lifetime of the open HDF5 file.
- Because the DEK+ format is stored together with the encrypted HDF5 file for the initial implementation, users typically specify only the encrypted HDF5 file during normal operation. Future implementations may additionally support external DEK+ objects.
- This workflow assumes that interactive terminal input is available. The user is prompted for their password only once when the file is opened, after which subsequent encryption and decryption operation proceed without further authentication.
- The procedure below illustrates the general sequence of operations performed when opening an encrypted HDF5 file using interactive password entry. Highlighted components indicate where user interaction occurs.



↓
Encrypt or Decrypt Data in the HDF5 File

Successful authentication results in recovery of the original DEK rather than generation of a new encryption key. Once recovered, the DEK remains resident within the `libgcrypt` secure memory pool for the lifetime of the open HDF5 file, allowing the Encryption VFD to perform subsequent encryption and decryption operations without repeatedly requesting the user's password.

2.1.5 Opening an Encrypted HDF5 File Without Terminal-Based Input

- This workflow performs the same DEK recovery process described in Section 2.1.3, except the password is supplied programmatically rather than through an interactive terminal prompt.
- As with the interaction workflow, the Encryption VFD automatically locates the associated DEK+ format and recovers the existing DEK without requiring the user to directly manipulate DEK+ metadata.
- The User will provide their desired password as part of the Encryption VFD configuration string. The user may use a trivial password by passing "" rather than passing a non-trivial password (i.e., "mykekpasswordgoeshere"). Because the password is supplied through the application configuration or API, it exists in ordinary application memory prior to KEK derivation. Consequently, this workflow should be reserved for trusted environments where interactive authentication is impractical or unavailable.
- The procedure below illustrates the general sequence of operations performed during non-interactive DEK recovery. Highlighted components indicate where application-provided input is consumed.

Open the Encrypted HDF5 File with `H5Fopen()`

↓
Load the DEK+ Format from the Second Page of the Encrypted HDF5 File

↓
Load the Password from the Configuration String

↓
Invoke the Password-Based KDF

↓
Derive the KEK

↓
Verify the Authentication Tag

↓
Recover the DEK

↓
Encrypt or Decrypt Data in the HDF5 File

From the perspective of the Encryption VFD, this workflow is functionally equivalent to the interactive recovery process. The only difference lies in how the password is supplied. Because the password is available in cleartext to the calling application before KEK derivation, users should carefully evaluate the security implications before selecting this workflow.

2.1.6 Password Management Overview

The password-management operations described in Sections 2.1.6 through 2.1.11 may be performed either through the `h5keytools` companion utility or through the corresponding public Encryption VFD key management API. Both interfaces invoke the same underlying implementation and therefore provide identical functionality and cryptographic behavior. Throughout the workflows below, terminal-based examples reference `h5keytools` for clarity, while non-terminal examples illustrate the equivalent programmatic API usage.

It is important to distinguish between changing a password and removing a password. Changing a password always preserves password-based protection by replacing one password-derived KEK with another. If a user wishes to continue using password-based protection while adopting an empty-string password, the **Change Password** operation should be used. In contrast, the **Remove Password** operation disables password-based protection entirely by removing the KEK and storing the DEK in cleartext within the DEK+ format. These operations therefore have different security properties and are intentionally treated as separate workflows.

2.1.7 Change password with Terminal-Based Input

- Changing the password does not generate a new Data Encryption Key (DEK). Instead, the existing DEK is recovered using the current password and immediately re-protected using a newly derived Key Encryption Key (KEK). This operation is commonly referred to as KEK rotation.
- Since the DEK itself remains unchanged, the encrypted contents of the HDF5 file do not need to be re-encrypted. Only the DEK+ format is rewritten with newly generated cryptographic material.

Invoke the Password Change Operation



Load the DEK+ Format from the Second Page of the Encrypted HDF5 File



Prompt the User for Their Current Password



Invoke the Password-Based KDF



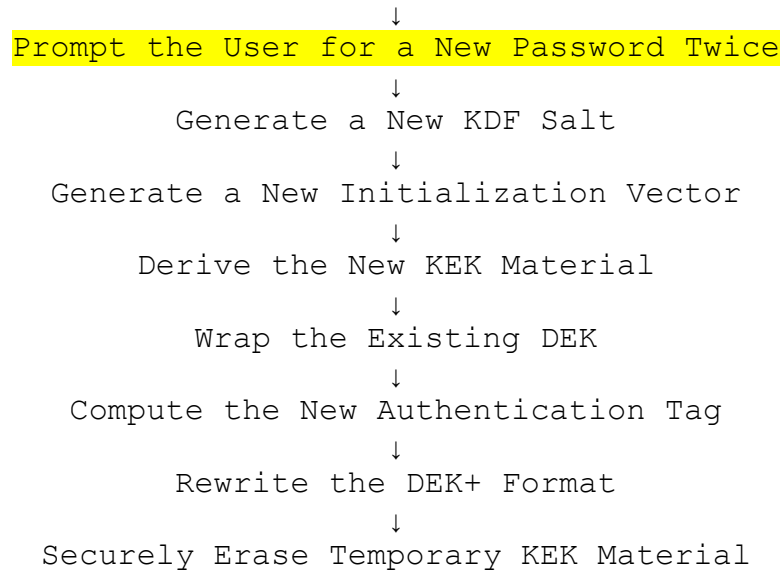
Derive the Existing KEK



Verify the Authentication Tag



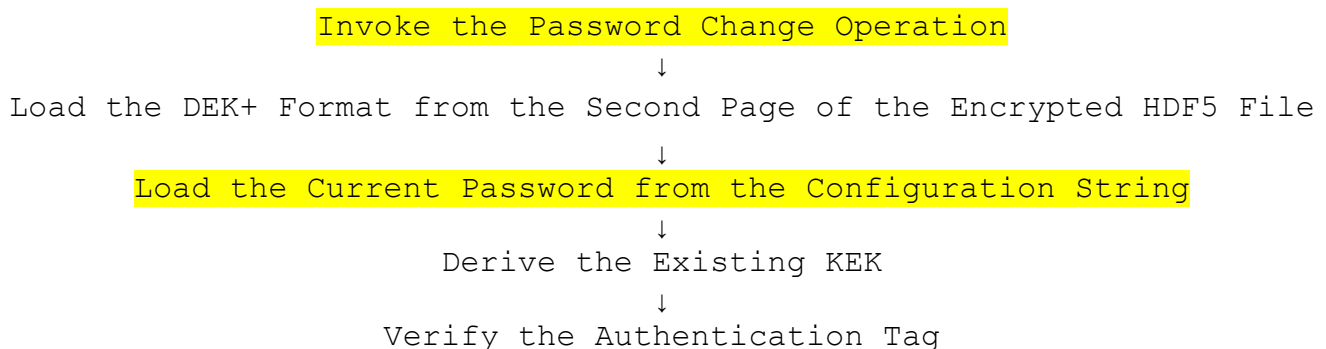
Recover the DEK into the libcrypt Secure Memory Pool

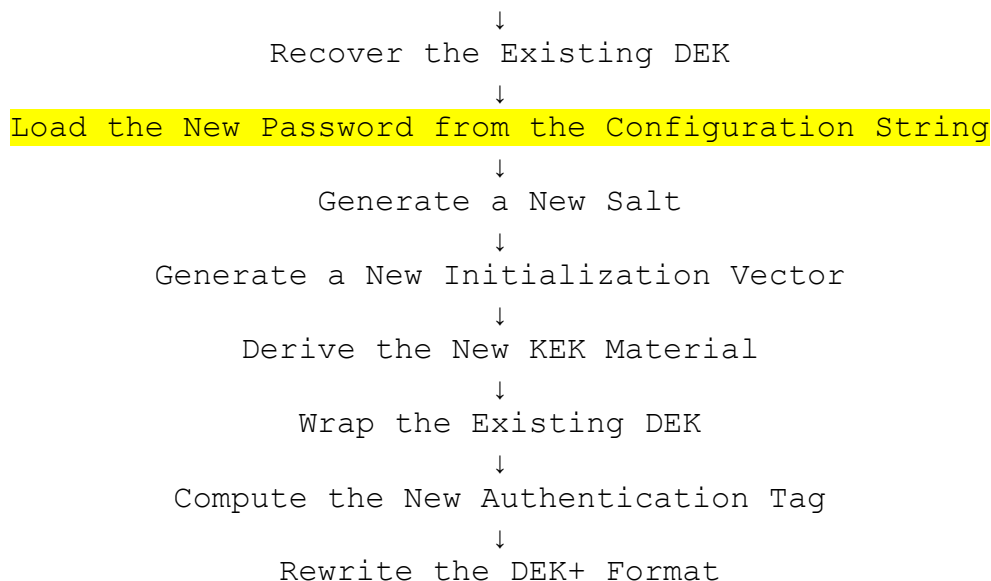


Although the password changes during this operation, the underlying DEK remains unchanged. Consequently, the encrypted HDF5 data does not require re-encryption. Instead, only the password-derived KEK, KDF salt, initialization vector, wrapped DEK, and authentication tag are regenerated and written back to the DEK+ format. This significantly reduces the cost of changing passwords, regardless of the size of the associated encrypted HDF5 file.

2.1.8 Change password without Terminal-Based Input

- This workflow performs the same KEK rotation operation as described in Section 2.1.5, except the existing and replacement passwords are supplied programmatically rather than being collected through an interactive terminal prompt.
- Password-management operations may be invoked either through the public Encryption VFD key management API or by h5keytools, both of which share the same underlying implementation and produce identical DEK+ output.
- The procedure below illustrates the general sequence of operations performed during a non-interactive password change. Highlighted components indicate where application-provided input is expected.

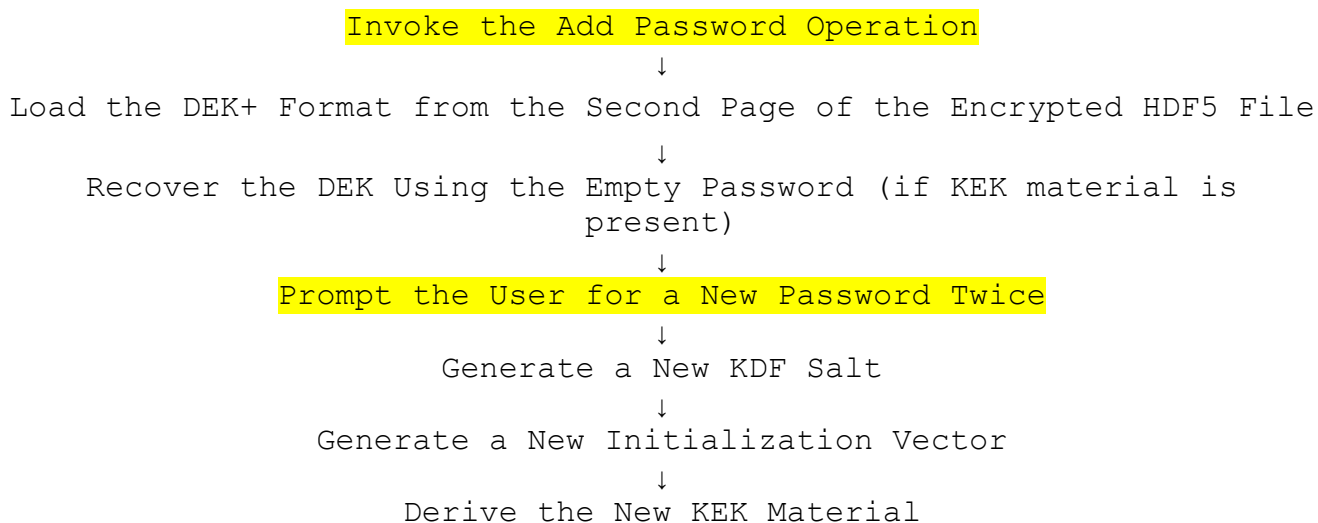


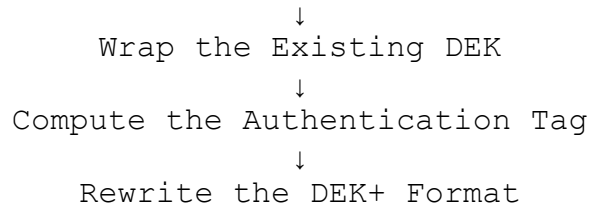


The workflow performs the same KEK rotation as the interactive process, except both the existing and replacement passwords are obtained through the public API rather than through a terminal prompt. Since the underlying DEK remains unchanged, the encrypted HDF5 file data do not require re-encryption.

2.1.9 Add a password with Terminal-Based Input

- Adding a password is intended for DEK+ objects that are currently stored without password protection. Rather than changing the underlying DEK, this operation introduces password-derived KEK protection for an existing DEK.
- Password-management operations may be performed through either h5keytools companion utility or the public Encryption VFD key management API. Both interfaces invoke the same implementation and therefore provide identical cryptographic behavior.

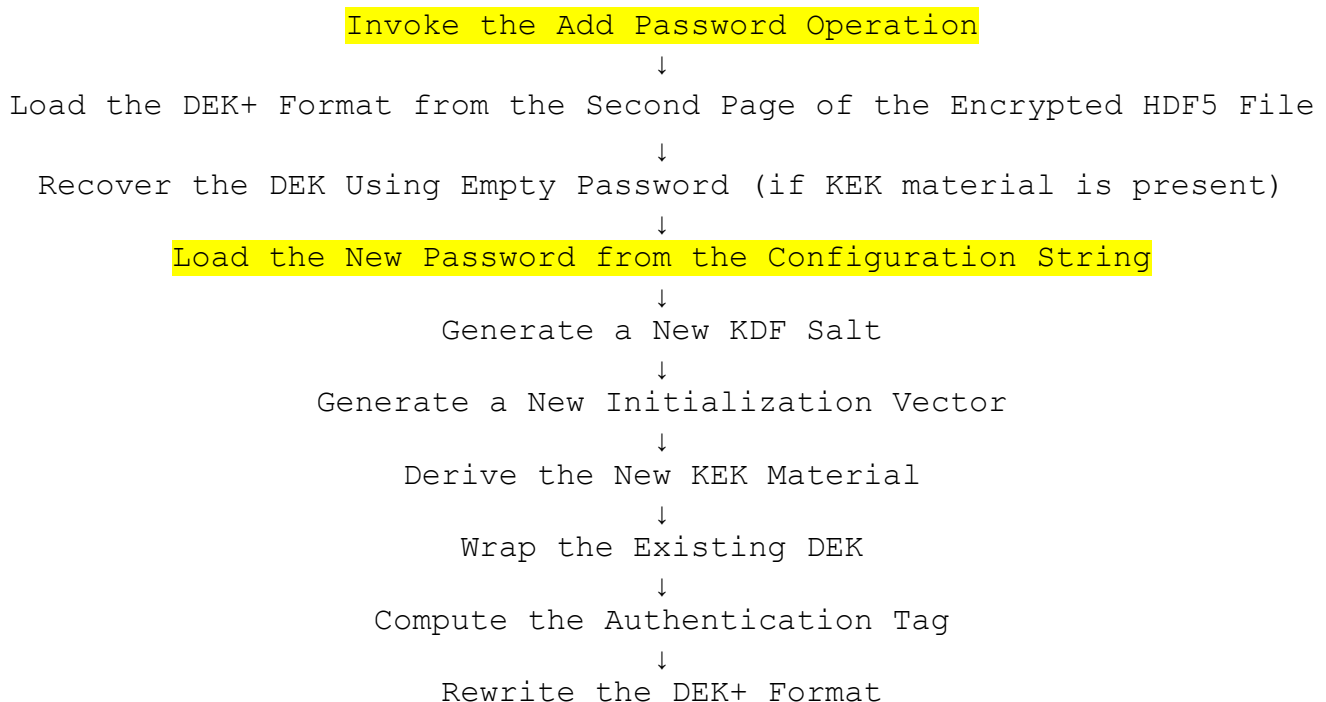




Adding a password restores password-based protection to an existing DEK+ format without altering the encrypted HDF5 data. Since only the KEK changes, the underlying DEK remains unchanged. The encrypted HDF5 file data does not require re-encryption.

2.1.10 Add a password without Terminal-Based Input

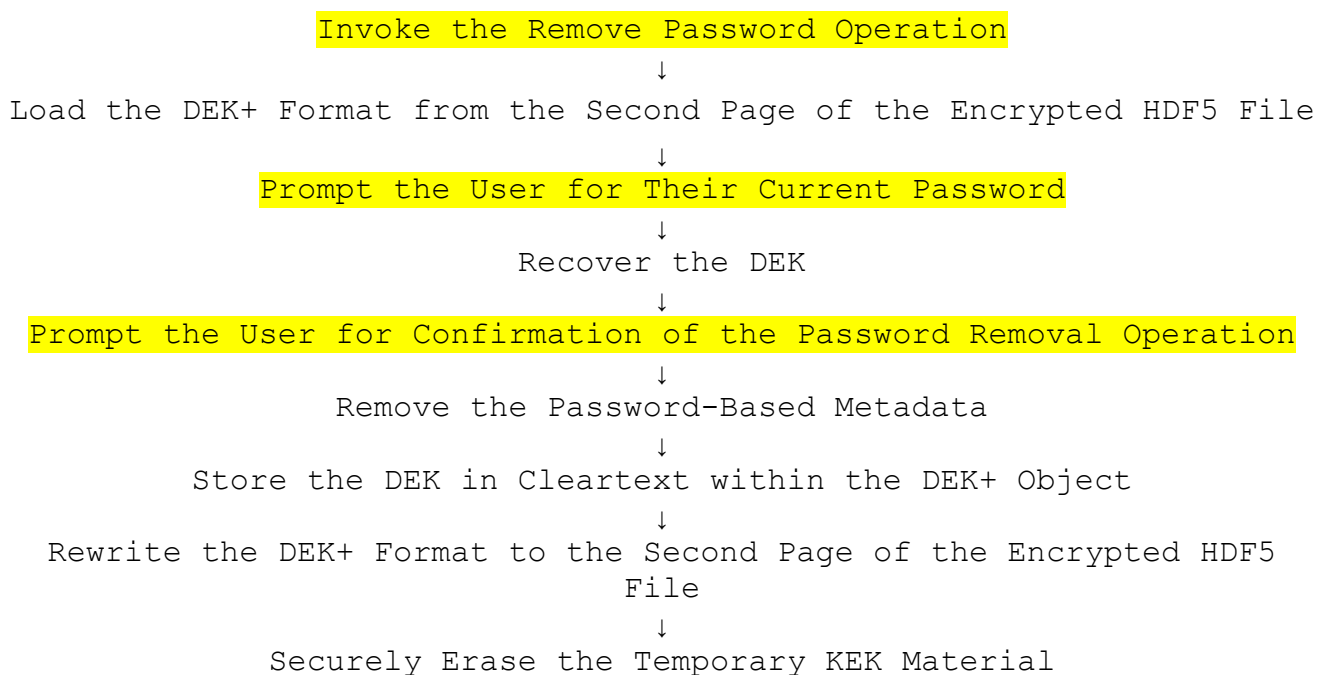
- This workflow performs the same password-addition operation described in Section 2.1.7, except the replacement password is supplied through the public API or configuration interface rather than through terminal prompts.
- Since only the KEK changes, the existing DEK and encrypted HDF5 data remain unchanged throughout the operation.
- The procedure below illustrates the non-interactive workflow. Highlighted components indicate where application-provided input is expected.



Adding a password restores password-based protection to an existing DEK+ format without altering the encrypted HDF5 data. Since only the KEK changes, the underlying DEK remains unchanged. The encrypted HDF5 file data does not require re-encryption.

2.1.11 Remove a password with Terminal-Based Input

- Removing a password permanently removes password-based protection from the DEK+ object. Unlike a password change, this operation does not derive a replacement KEK. Instead, the recovered DEK is subsequently stored in cleartext within the DEK+ format.
- Because this operation substantially weakens the protection provided by the DEK+ format, it should require explicit user confirmation and should only be available when the appropriate security policy permits cleartext DEKs.
- The procedure below illustrates the interactive password-removal workflow. Highlighted components indicate where user interaction is expected.

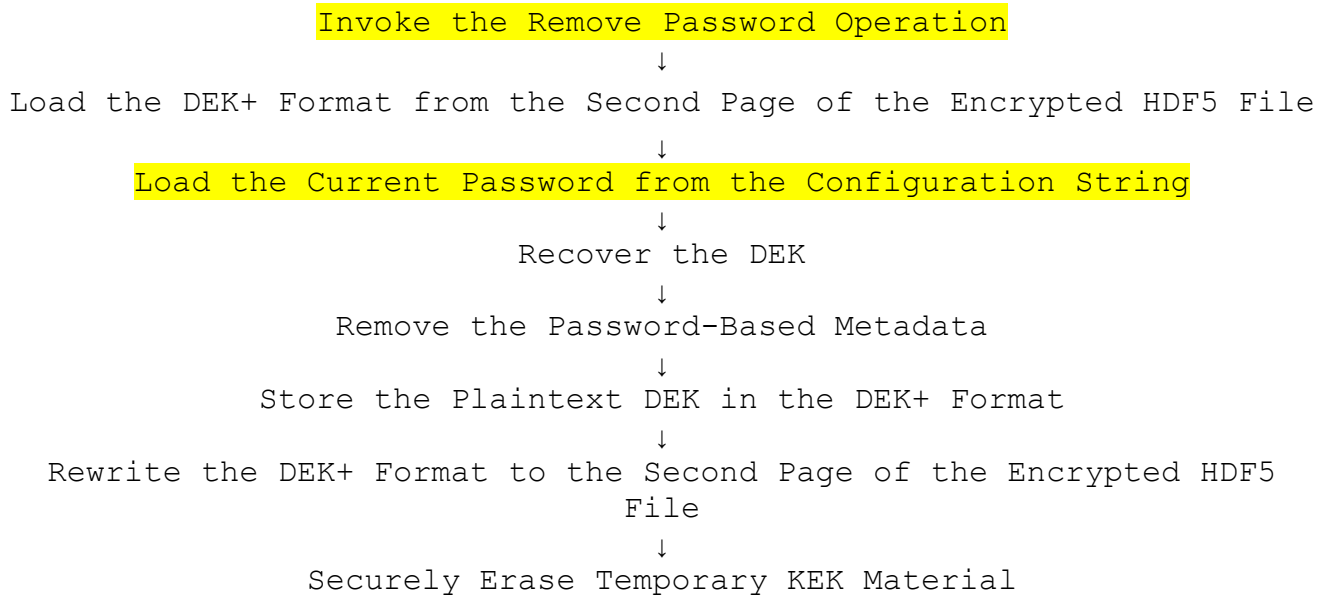


Removing the password intentionally changes the DEK+ security model by eliminating password-based protection entirely. Following successful authentication and user confirmation, the recovered DEK is written directly into the DEK+ format without deriving a replacement KEK. Consequently, this workflow should be reserved for trusted systems or specialized environments where cleartext DEK storage is explicitly permitted by policy.

2.1.12 Remove a password without Terminal-Based Input

- This workflow performs the same password-removal operation described in Section 2.1.9, except the current password is supplied programmatically through the public API.
- Upon successful authentication, the recovered DEK is written to the DEK+ format in cleartext and all password-derived KEK material is removed from the object.

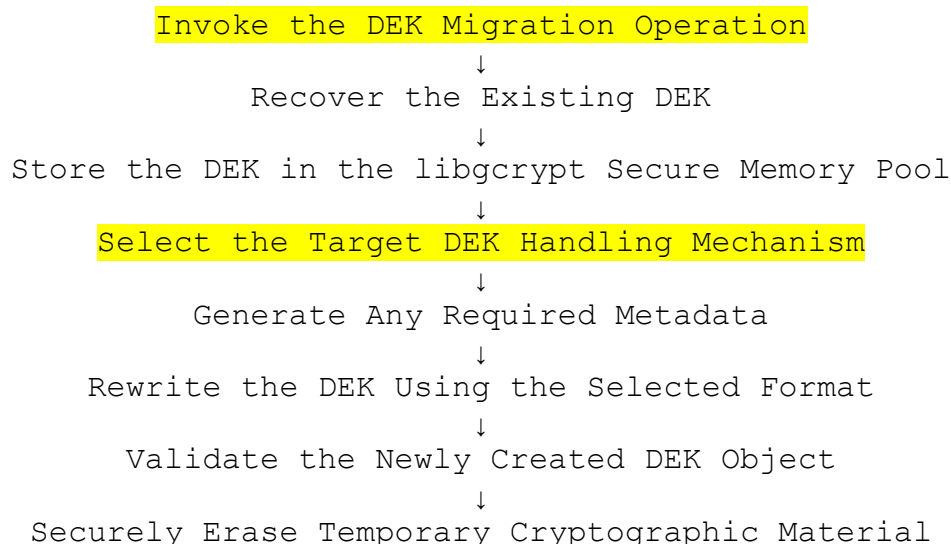
- The procedure below illustrates the non-interactive workflow. Highlighted components indicate where application-provided input is expected.



Removing the password intentionally changes the DEK+ security model by eliminating password-based protection entirely. Following successful authentication, the recovered DEK is written directly into the DEK+ format without deriving a replacement KEK. Consequently, this workflow should be reserved for trusted systems or specialized environments where cleartext DEK storage is explicitly permitted by policy.

2.1.13 Migrating Between Supported DEK Handling Mechanisms

- Although the initial implementation focuses on the DEK+ format, the overall Encryption VFD architecture is intentionally designed to support multiple DEK handling mechanisms. Future releases will therefore permit users to migrate between supported key storage mechanisms without requiring re-encryption of the associated HDF5 data.
- Regardless of the selected storage mechanism, the Encryption VFD ultimately consumes the same recovered 32-byte DEK. Consequently, migration consists only of recovering the existing DEK and rewriting that same DEK using a different supported representation. The encrypted HDF5 data remain unchanged throughout the operation.
- This functionality is not expected to be included in the initial implementation, but is anticipated to become available interactively through the `h5keytools` companion utility and programmatically through public API calls as additional DEK handling mechanisms are introduced.
- The procedure below illustrates the conceptual migration workflow. Highlighted components indicate where user or application interaction may occur.



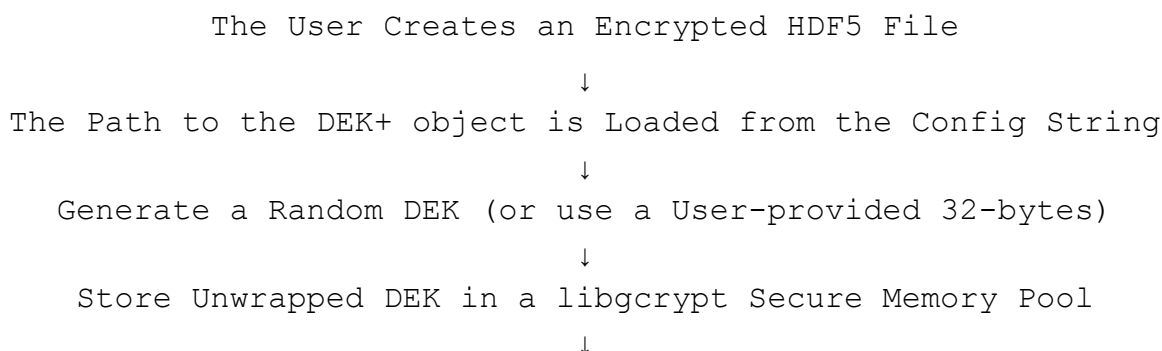
Because the Encryption VFD always operates on the recovered plaintext DEK rather than on the underlying storage representation, migration between supported DEK handling mechanisms does not require re-encryption of the protected HDF5 data. Instead, the existing DEK is recovered into the `libgcrypt` secure memory pool and immediately rewritten using the selected output mechanism. This capability is planned for a future release and is expected to be implemented through the `h5keytools` companion utility and public API calls using the same public key management interfaces described throughout this document.

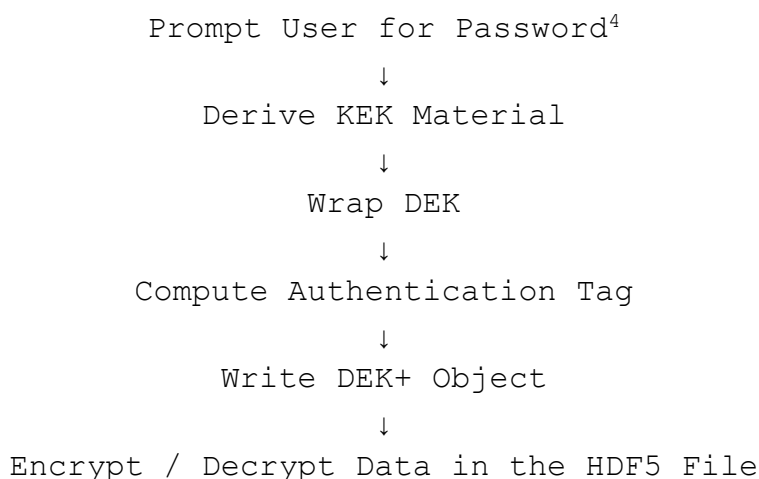
3 Implementation Outline

This section focuses on outlining the individual components of the local key generation solution.

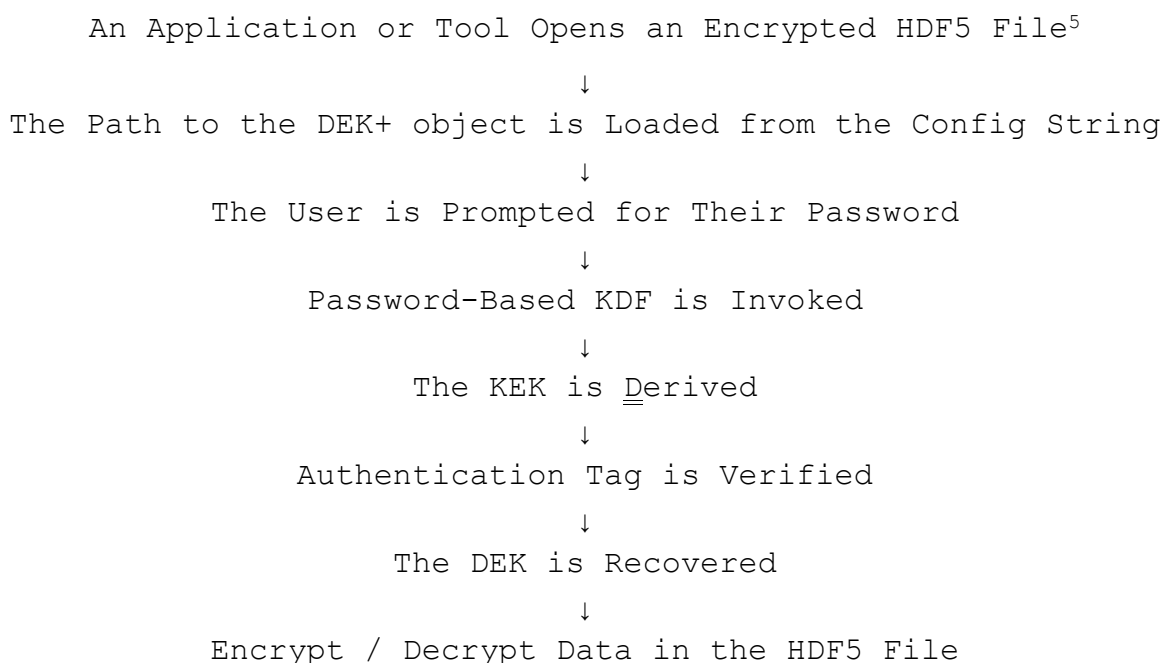
3.1 High-Level Usage Description

There are several major components necessary to facilitate the implementation and use of the local key generation solution. To illustrate each of these pieces, it is first important to establish what the typical workflow will look like when an HDF5 file is created using the Encryption VFD:





Similarly, when an HDF5 file is interacted with that requires the Encryption VFD:



In both situations, it is important to note that the KEK is derived in memory and only retained long enough to recover the DEK. In contrast, the decrypted DEK will be stored in the `libgcrypt` secure memory pool until the file is closed (or an error causes the application/script to crash). This secure

⁴ If a user does not wish to associate a password with the KEK, they may simply press **Enter** when prompted. This results in an empty password (``'') being used during KEK derivation and causes subsequent executions to bypass the password prompt when permitted by the Encryption VFD configuration. This mode should only be used when interactive password entry is unavailable or otherwise undesirable, as it intentionally removes the password-based protection provided by this design.

⁵ Interaction may occur through an application, script, command-line utility, or another HDF5-compatible tool that utilizes the Encryption VFD.

retention removes the need to recover the DEK each time it is needed to process a page, while maintaining the security of the on-disk DEK. This also provides a minimal surface for user interaction. When a new data file is created or an existing data file is opened, the user is prompted for their password at most one time.

3.2 DEK+ Format Summary

To accommodate the local key generation solution, a simple DEK+ format structure will be adopted. This DEK+ format, serialized as a binary representation, will consist of two main components; a fixed-length portion and a variable-length portion. The fixed-length portion consists of a fixed binary header represented in memory by a defined C structure. The header contains the DEK+ format version, serialized-state flags, KDF parameters, cryptographic algorithm identifiers, field lengths, and other metadata required to interpret the remainder of the DEK+ object. Note that this information is derived and is not intended to be manually interacted with by the user. In the variable-length portion of the DEK+ object, the header, KDF salt, IV, wrapped DEK, and authentication tag will be stored. Taken as a whole, this DEK+ object encompasses the information necessary for recovering the decrypted DEK. For a detailed discussion, see section 4.2 of this document.

3.3 Password Usage

The local key generation solution is built around the necessity of requiring a user to secure their DEK through a password-derived KEK. What this means in practice is a user will be prompted for a password when creating a new file or interacting with an existing file through the Encryption VFD. When this user-provided password is combined with other elements stored in the DEK+ format, an attack-resistant, wrapped DEK is produced. In situations where the password prompt is inaccessible or undesirable, the user may invoke `h5keytools` to convert⁶ a DEK+ object into a representation that no longer requires a non-empty password. While removing the password-based protection for the wrapped DEK, this option is provided primarily for non-interactive environments, trusted isolated systems, and HPC workflows where a terminal prompt is unavailable. It is recommended that after these tests are completed, the user rekey the KEK with a password to ensure security of the DEK.

During interactive operations, the user will be prompted for their password through the terminal using an input routine that suppresses character echoing. This prevents the password from being displayed on the terminal and inadvertently captured in terminal logs or command histories. The entered password temporarily exists in cleartext, is stored in the `libgcrypt` secure memory pool, and is only retained long enough to derive the KEK material and is never written to disk or stored within the DEK+ format.

Once the password has been collected, it is immediately supplied to the configured Key Derivation Function (`scrypt` for the initial implementation) together with the KDF salt stored in the DEK+ object. The KDF produces 64 bytes of KEK material, with the first 32 bytes used as the AES-256-CBC wrapping key and the remaining 32 bytes used as the HMAC-SHA256 authentication key. After

⁶ The rekey procedure, which is outlined later in this document, is a procedure used to change the KEK while preserving the DEK. This distinction is important because it allows users to change their password, and therefore the password-derived KEK, without requiring the associated HDF5 data to be re-encrypted. Functionally, the rekey operation performs a rotation of the KEK while leaving the DEK unchanged.

successful key derivation, the plaintext password buffer should be securely overwritten before being released, minimizing the amount of time that the password remains resident in application memory.

Since the derived KEK material constitutes secret cryptographic material, it should be allocated from the `libgcrypt` secure memory pool. Unlike the recovered DEK, however, the derived KEK exists only for the duration required to authenticate the DEK+ object and recover or rewrap the DEK. Once these operations are complete, the KEK material should be securely erased.

3.4 User-Interaction and Policy Model

The implementation distinguishes between three independent layers of policy. Build-time capabilities determine which features are compiled into the resulting binary. Runtime policy determines which of those capabilities are authorized for a particular operation. Serialized DEK+ header flags describe the persistent representation stored within an existing DEK+ object. These layers are independent and must all be mutually consistent before a security-sensitive operation is performed.

Users are not expected to construct the DEK+ operation-flag bitmasks or modify the DEK+ object header flags during ordinary use. Instead, the user selects a key management operation through the `h5keytools` companion utility or through the public key management API. The selected interface translates the requested behavior into the internal operation code, runtime policy values, and operation flags required by the shared DEK+ implementation. Build-time options dictate which security-sensitive capabilities are available in the resulting library and companion utility. Examples include support for empty-string passwords, cleartext DEK storage, cleartext DEK exportation, or the creation of external DEK+ copies (as future work). Enabling one or more build-time options makes the corresponding implementation available, but does not automatically request or authorize its use during normal execution. Further, enabling one or more of these build-time options will generate a corresponding compiler warning pertaining to the security implications of allowing for a particular feature or action.

When the user invokes an operation, the selected front-end derives the appropriate runtime controls for the requested behavior. For example, an interactive password-change command permits terminal input, while a programmatic password-change request bypasses terminal prompting because the current and replacement passwords are supplied through the public API. Similarly, a request to remove password protection causes the implementation to request the relevant security-reducing operation and, where applicable, collect explicit confirmation from the user.

The shared DEK+ implementation validates three independent conditions before performing a security-sensitive operation:

- The requested capability must be present in the current build.
- The requested operation and supplied runtime policy must authorize the behavior.
- The existing DEK+ representation, user input, and requested output representation must be mutually consistent.

A runtime operation flag cannot enable functionality that was excluded at build time. If a caller requests an unavailable capability, the operation fails before the DEK+ object is modified or plaintext key material is exposed. The resulting error will identify the unsupported capability and, where

appropriate, indicate which build-time option is required to successfully complete the desired operation.

DEK+ object header flags are generated and interpreted internally. They describe the persistent representation of the DEK+ object, such as whether the object uses an empty-string password or stores a cleartext DEK. Users do not set these flags directly. After validating the requested operation, runtime policy, and build capabilities, the implementation derives and writes the appropriate header flags when creating or rewriting the DEK+ object.

3.5 Usage with the Encryption VFD

From the perspective of the Encryption VFD, the local key generation solution serves as a key provider rather than an encryption mechanism. The responsibility of this component is limited to securely creating, recovering, and maintaining the DEK within the `libgcrypt` secure memory pool. Once recovered, the DEK is consumed by the existing Encryption VFD encryption routines without requiring modifications to the underlying data encryption or decryption logic. This separation of responsibilities minimizes the impact of introducing additional key management architectures in future releases while maintaining a consistent interface for the Encryption VFD.

3.6 H5keytools Companion Utility

To support, and simplify, migration between supported key management mechanisms, a companion utility named `h5keytools` is proposed. The purpose of this utility is not to generate new encryption keys, but rather to recover an existing DEK using one supported mechanism and rewrite the same DEK using a different supported mechanism⁷. This will allow users to migrate between key management strategies without requiring re-encryption of the associated HDF5 data (i.e., the DEK is preserved).

3.6.1 Purpose

The intent of `h5keytools` is to provide a single interface for manipulating Encryption VFD key material regardless of the underlying storage mechanism. Rather than exposing multiple independent utilities for each supported format, the tool serves as a centralized mechanism for creating, converting, validating, and rekeying DEKs while preserving a consistent user interface and minimizing opportunities for incorrect usage.

The `h5keytools` utility is intended to invoke the same public DEK+ format interfaces implemented for the Encryption VFD rather than maintaining an independent implementation of the key management algorithms. This ensures that both components interpret the DEK+ format identically, minimizes duplicated cryptographic code, and guarantees that future changes to the DEK+ format are reflected consistently across all supported tools.

Although the Encryption VFD and `h5keytools` expose different user-facing workflows, both components should rely on the same underlying DEK+ implementation. Shared routines should cover

⁷ This process, despite the conceptual overlap, should be treated as fundamentally different than rekeying a KEK.

DEK generation, DEK wrapping/unwrapping, HMAC computation and verification, header validation, serialization, deserialization, and secure-memory cleanup.

3.6.2 Supported Operations

The operations supported by the `h5keytools` utility are outlined in this section. For the initial release, the following functionality will be built out.

3.6.2.1 Create DEK

The `h5keytools` utility may be used to create a new 32-byte binary DEK for use with the Encryption VFD. This operation is intended for users who require a cleartext DEK for workflows outside of the DEK+ format, such as passing the DEK directly through an Encryption VFD configuration structure, configuration string, or an external DEK+ file (as potential future work). Because this output is not password protected, the generated DEK should be handled as sensitive key material immediately after creation. If written to disk, the resulting file should be treated as equivalent to the unwrapped encryption key for all data protected by that DEK.

3.6.2.2 Create DEK+ Object

The `h5keytools` utility may create a new DEK+ object by generating or importing a 32-byte binary DEK, prompting the user for a password, deriving KEK material, wrapping the DEK, computing an authentication tag, and writing the resulting DEK+ format to the requested output location. For the initial implementation, this operation is expected to support storing the DEK+ format in a reserved page of an encrypted HDF5 file. Future support may allow for the DEK+ format to utilize an external DEK+ object.

Similar DEK+ creation behavior may also occur inside the Encryption VFD when a new encrypted HDF5 file is created and the selected key management mode requires a DEK+ object. In that case, the Encryption VFD may generate the DEK, prompt for a password, create the DEK+ object, and retain the recovered DEK in a `libgcrypt`-based secure memory pool for immediate use. To avoid divergent behavior, the Encryption VFD and `h5keytools` should share the same DEK+ creation, wrapping, authentication, validation, and serialization routines.

3.6.2.3 Validate DEK+ Object

The `h5keytools` utility should provide a validation operation for DEK+ objects. At minimum, validations should confirm that the DEK+ header is well formed, the format version is supported, the expected lengths are present, algorithm identifiers are valid, and the total file size is consistent with the declared format. When a password is supplied, validation may also derive the KEK material and verify the HMAC-SHA256 authentication tag. Structural validation confirms that the DEK+ object is well formed. Authentication validation confirms that the HMAC-SHA256 authentication tag is valid.

3.6.2.4 Change Password

Changing the password is fundamentally a KEK rotation operation. The DEK remains unchanged, while the password-derived KEK material, KDF salt, IV, wrapped DEK, and authentication tag are regenerated. The user provides the current password to recover the existing DEK, then provides a new password to rewrap that same DEK. This operation should be performed by `h5keytools`, not

by the normal Encryption VFD open path. The Encryption VFD may unlock and consume the DEK, but password changes should be handled by the companion utility.

3.6.2.5 Remove Password

Removing a password converts a password-protected DEK+ object into a DEK+ object with no password protection. The DEK remains unchanged, but the DEK is unwrapped using KEK material derived from the provided password and then stored as cleartext. This operation should require explicit user confirmation and should only be permitted when saving a cleartext DEK is enabled. Removing the password substantially weakens the security of the DEK+ object and therefore should only be permitted when both:

- “the implementation has been built with cleartext-DEK support enabled, and”
- “the current runtime policy explicitly authorizes password removal.”

Interactive use should additionally require explicit user confirmation before the DEK+ object is rewritten without a password.

3.6.2.6 Add Password

Adding a password converts a DEK+ object that stores a cleartext DEK into a password-protected DEK+ object. The cleartext DEK loaded from the DEK+ object and the DEK is wrapped using KEK material derived from the newly supplied password (empty or otherwise) and a newly generated salt. This operation restores password-based protection to a cleartext DEK without requiring re-encryption of the associated HDF5 data.

3.6.2.7 Update KDF parameters

The `h5keytools` utility may be used to update KDF parameters for an existing DEK+ object. This operation recovers the DEK using the existing password and KDF parameters, then rewraps the same DEK using newly selected KDF parameters. For example, a future implementation may increase the `script` work factor to improve the resistance against offline password-guessing attacks. Updating KDF parameters associated with `script` directly affects the work factor, and by extension, the cost of DEK recovery. Altering the work factor parameters does not alter the DEK or require re-encryption of the associated HDF5 data.

3.6.2.8 Future Format Migration

Future versions of the DEK+ format may introduce new header fields, additional authentication metadata, alternate KDFs, alternate wrapping algorithms, or support for multiple wrapped DEK records. The `h5keytools` utility should provide the primary migration path between supported DEK+ versions. A migration operation should recover the DEK using the existing format, then write

the same DEK using the target format. Older implementations should reject unsupported versions rather than attempting to interpret unknown layouts.

3.6.3 Transferring Between Supported DEK Handling Mechanisms

For the initial implementation, the following DEK handling mechanisms are expected to be supported:

- A cleartext DEK is supplied directly through the Encryption VFD configuration structure or configuration string.
- A cleartext binary DEK is stored in an external file (without any additional content in this file).
- The DEK+ format described in this document using a password-derived KEK.
- The DEK+ format using an empty string password-derived KEK when explicitly permitted by the Encryption VFD configuration⁸.
- The DEK+ format using no password string and no KEK (resulting in a cleartext DEK being stored) when explicitly permitted by the Encryption VFD configuration⁹.

Note: There has been internal discussion as to whether the DEK should be stored in one of the first pages of the encrypted HDF5 file. While this discussion is ongoing, it is important to note that the cleartext DEK and the proposed DEK+ format may also eventually be stored in an HDF5 file rather than a separate file. This addition won't directly change the remainder of the discussion, but does add additional supported key storage mechanisms and will need to be supported by the proposed tool.

Because all supported mechanisms ultimately provide the same 32-byte binary DEK to the Encryption VFD, converting between formats requires only recovering the DEK into the `libgcrypt` secure memory pool and rewriting it using the desired output format. At no point is the associated HDF5 data modified, as the DEK itself remains unchanged throughout the conversion process. Fundamentally, three conversions should be considered as described below.

3.6.3.1 Convert from Secure to Secure

Converting between secure storage mechanisms (for example, changing the KEK password or migrating between different DEK+ object instances) should be considered a standard operation. During these operations, the DEK is recovered into the `libgcrypt` secure memory pool, re-protected using the requested output format, and immediately removed from any temporary buffers once the conversion completes.

⁸ ~~Note that from a user-defined perspective, the cleartext binary DEK external file and the DEK+ format using an empty string password are identical. When using the an empty string password-enabled DEK+ format, the terminal prompt may be bypassed. The sole difference lies in whether the DEK may be recovered directly (from the cleartext binary) or requires a decryption operation (while trivial due to the absence of an actual password, the KEK material must still be recovered to unwrap the DEK).~~

⁹ In this case, the stored DEK is not wrapped and stored as unsecured cleartext.

3.6.3.2 Convert from Insecure to Secure

When converting from a cleartext DEK supplied by the user to the DEK+ format, `h5keytools` will assume that the supplied DEK is already cryptographically suitable for use with AES-256 (at least for the initial release). Since the utility cannot determine how the key was originally generated, it is the responsibility of the user to ensure that the supplied DEK was generated using a cryptographically secure random number generator.¹⁰ Imposing a predictable, reused, or otherwise weak DEK into the DEK+ format protects the key only while it is stored at rest; it cannot improve the entropy or security of the underlying DEK itself.

3.6.3.3 Convert from Secure to Insecure

Exporting a DEK from a protected format into a cleartext representation fundamentally changes the security model of the Encryption VFD. Although the recovered DEK exists only within the `libgcrypt` secure memory pool during normal operation, exporting the DEK necessarily copies it into ordinary application memory before writing it to persistent storage. Once written as a cleartext binary file or embedded directly within an Encryption VFD configuration, the DEK is no longer protected by the password-derived KEK, the authenticated DEK+ format, or the `libgcrypt` secure memory pool subsystem.

From this point forward, any process, user, backup system, or adversary capable of reading the exported DEK obtains the complete encryption key required to decrypt all data protected by the DEK. Unlike attacks against the DEK+ format, which require successful password recovery through the KDF, attacks against a cleartext DEK require only access to the exported key itself. Exporting the DEK permanently bypasses every protection provided by the DEK+ format. Once exported, possession of the cleartext DEK alone is sufficient to decrypt all data protected by the key. Password recovery, KEK derivation, authentication-tag verification, and all other DEK+ security mechanisms are no longer involved.

3.6.4 Build-Time Protection

To discourage accidental removal of password protection, support for exporting plaintext DEKs should not be compiled into the Encryption VFD by default. Instead, the functionality should be conditionally compiled through a dedicated build-time macro (for example, `H5FD_ALLOW_CLEARTEXT_KEY_EXPORT`). When enabled, the compiler should emit a warning indicating that the resulting binary contains functionality capable of exporting plaintext DEKs from the `libgcrypt` secure memory pool. Additionally, the `h5keytools` utility should present a runtime warning requiring explicit user confirmation before writing a cleartext DEK to persistent storage.

As an additional safety measure, `h5keytools` should refuse to overwrite an existing output key by default. Since the utility operates directly on encryption keys, unintentionally replacing an existing key may permanently prevent recovery of the associated HDF5 data. Consequently, overwriting an existing output file should require an explicit command-line option (for example, `--force` or `--`

¹⁰ Should a user wish to avoid this responsibility, they should consider using the password-based key solution proposed in this document and allow `libgcrypt` to create a DEK on their behalf.

overwrite) and should generate an additional warning before proceeding. This behavior provides consistency with the overall philosophy of the proposed implementation, which favors secure defaults while requiring explicit user acknowledgement before performing potentially destructive operations.

4 Theory of Operations

In an effort to not conflict with other Encryption VFD documentation, this section will focus on the general operations that will interact with the local key generation solution. Further interactions may be added as necessary.

4.1 Create a New DEK+ Object

1. The Encryption VFD determines that a new DEK+ object must be created.
2. The user is prompted for a password (or an empty-string password if permitted, assuming the appropriate flags are set/provided).
3. A random 32-byte DEK is generated and stored in the `libgcrypt` secure memory pool (if the user does not supply a 32-byte DEK and the appropriate flag is not specified).
4. A random salt and IV are generated.
5. The KEK material is derived using `scrypt`.
6. The DEK is wrapped using `AES-256-CBC`.
7. An `HMAC-SHA256` authentication tag is computed.
8. The DEK+ format is written to file automatically (either to an external file or within the second page of an HDF5 file created through the Encryption VFD).
9. Temporary KEK material is securely erased.
10. The DEK remains resident in `libgcrypt` secure memory pool for use by the Encryption VFD until the HDF5 file is closed or until an unrecoverable error causes the Encryption VFD to shut down and release the `libgcrypt` secure memory pool.

4.2 Interacting with an Existing Encrypted File

1. The Encryption VFD determines that an encrypted key already exists for the target HDF5 file.
2. The DEK+ format header is read and validated to ensure that the file format and cryptographic parameters are supported.
3. If required by the DEK+ format flags and Encryption VFD configuration, the user is prompted for their password through the terminal. Empty-string password operation is permitted only when both the DEK+ format metadata and the current Encryption VFD configuration explicitly allow it.
4. The KEK material is reconstructed using the stored KDF parameters, KDF salt, and user-provided password (or empty password if set/allowed). If the DEK+ object carries the

`H5FD_DEKPLUS_FLAG_CLEARTEXT_DEK` flag, the KEK derivation, authentication tag verification, and DEK unwrapping are bypassed because the DEK is already stored as cleartext and does not require recovery.

5. The `HMAC-SHA256` authentication tag is recomputed and verified before any decryption operation is performed.
6. Upon successful authentication, the wrapped DEK is decrypted using `AES-256-CBC` and stored in the `libgcrypt` secure memory pool.
7. Temporary KEK material and other intermediate cryptographic buffers are securely erased from memory.
8. The recovered DEK remains resident in the `libgcrypt` secure memory pool for the lifetime of the HDF5 file and is used by the Encryption VFD for subsequent encryption and decryption operations.

4.3 Rotating the KEK using `h5keytools`

1. The user invokes `h5keytools` with a rekey operation.
2. The existing DEK+ object is opened and validated, and the user is prompted for their current password through the terminal.
3. The existing KEK material is reconstructed, the authentication tag is verified, and the wrapped DEK is successfully recovered into the `libgcrypt` secure memory pool.
4. The user is prompted for a new password. If an empty password is requested, the operation proceeds only when `allow_empty_key_password` is enabled.
5. A new random KDF salt and initialization vector are generated.
6. A new 64-byte KEK is derived using the updated password and newly generated KDF parameters.
7. The existing DEK is wrapped using the newly derived KEK, a new `HMAC-SHA256` authentication tag is computed.
8. The DEK+ object is atomically written using the updated header, salt, IV, wrapped DEK, and authentication tag.
9. Temporary KEK material is securely erased, while the unmodified DEK remains available in the `libgcrypt` secure memory pool for continued Encryption VFD operation.

5 Technical Details

This section expands upon the implementation outline presented earlier in this document by describing the proposed DEK+ format, public interfaces, internal helper functions, and execution flow necessary to integrate password-based key management into the Encryption VFD. The intent is not to provide a complete implementation specification, but rather to define the architectural components, interfaces, and cryptographic responsibilities required to produce a consistent implementation.

Where appropriate, the functionality described throughout this section is expected to be shared between the Encryption VFD and the `h5keytools` companion utility to minimize duplicated logic and ensure consistent interpretation of the DEK+ format.

5.1 Encryption VFD Overview

As of the time of writing, the Encryption VFD allocates a secure memory pool using `libgcrypt` during the initialization phase. In short, a `libgcrypt` secure memory pool is a heap of memory that is locked to the system RAM and is forbidden from being written to disk. Additionally, a `libgcrypt` secure memory pool prevents overflow-based exploitation techniques. As such, unwrapped keys are stored in this memory pool. Additionally, using the version of the Encryption VFD available at commit tag `14ca76b8072351f0bd4270a29c5958489702763c`, two distinct methods are currently supported for passing in a cleartext key:

1. The user provides an unwrapped, DEK directly into the Encryption VFD through a File Access Property List (FAPL):
 - i. The DEK may be provided as part of the `H5FD_crypt_vfd_config_t` structure
 - ii. The hex-encoded DEK can be set using the Encryption VFD configuration string
2. The DEK is stored in a separate file as a binary blob with no additional contents. To access this DEK, the file pathing is provided through one of the following methods:
 - i. As part of an instance of the `H5FD_crypt_config_t` structure
 - ii. As part of the Encryption VFD configuration string
 - iii. Using the `H5FD_CRYPT_KEY_PATH` environment variable

As of the time of writing, a User may freely switch between the method they provide the DEK. The introduction of the DEK+ format will retain this freedom, with the understanding that the user may need to recover the cleartext version of the wrapped DEK from the DEK+ format should they wish to provide the DEK through a different supported mechanism. Independent of the key distribution strategy, the Encryption VFD will be responsible for maintaining the functionality of methods for loading in a DEK.

5.2 DEK+ Format

The DEK+ format is an application-defined binary format used to store the information necessary for recovering a 32-byte binary DEK. A DEK+ object may be stored as an external DEK+ object or, in future implementations, within a reserved page of an encrypted HDF5 file created through the Encryption VFD. The DEK+ format does not store the cleartext DEK (unless configured to do so), the user password, or the derived KEK material. Instead, it stores the metadata and encrypted key material required to reconstruct the KEK from the user password, authenticate the DEK+ contents, and unwrap the DEK.

Common general-purpose private-key containers such as PKCS#8, PKCS#12, and PEM are not used because this design protects a symmetric DEK rather than an asymmetric private key or certificate chain. A compact application-defined binary format is sufficient for the initial implementation and

allows the Encryption VFD and `h5keytools` to share a single implementation of the DEK+ format serialization, validation, wrapping, and recovery.

5.2.1 DEK+ Object Layout

Version 1 of the DEK+ format consists of a fixed-size header followed by fixed-size payload fields. Although the payload fields are described as variable-length in the header for future compatibility, the initial implementation utilizes exact field sizes.

Layout: DEK+ Format

byte	byte	byte	byte
Magic – fixed file identifier			
Three bytes Reserved for Future Use			Version
Flags			
KDF ID			
KDF N			
KDF r			
KDF p			
KDF output length			
Cipher ID			
Cipher mode			
MAC ID			
DEK length			
Salt length			
IV length			
IV length			
Wrapped DEK length			
MAC length			
KDF salt (16 bytes in the initial implementation)			

...
IV (16 bytes in the initial implementation)
...
Wrapped DEK (48 bytes in the initial implementation)
...
Authentication tag (32 bytes in the initial implementation)
...

Fields: Content of the DEK+ Format

Field	Size	Description
Magic	8 bytes	Fixed file identifier, expected to be <code>EVFDKEY\0</code> .
Reserved	3 bytes	Reserved for future use. Must be written as zero and ignored on read unless defined by a future version.
Version	1 byte	DEK+ format version. Version 1 indicates the initial implementation.
Flags	4 bytes	Bitmask describing the DEK+ policy and optional behavior.
KDF ID	4 bytes	Identifier for the password-based KDF. Version 1 uses <code>scrypt</code> .
KDF N	4 bytes	<code>scrypt</code> CPU/memory cost parameter. Version 1 default is 32768.
KDF r	4 bytes	<code>scrypt</code> block-size parameter. Version 1 default is 8.
KDF p	4 bytes	<code>scrypt</code> parallelization parameter. Version 1 default is 1.
KDF output length	4 bytes	Number of bytes produced by the KDF. Version 1 default is 64.
Cipher ID	4 bytes	Identifier for the cipher used to wrap the DEK. Version 1 uses AES-256.
Cipher mode	4 bytes	Identifiers for the cipher mode. Version 1 uses CBC.
MAC ID	4 bytes	Identifier for the authentication algorithm. Version 1 uses HMAC-SHA256.
DEK length	4 bytes	Length of the unwrapped or cleartext DEK. Version 1 requires 32.

Salt length	4 bytes	Length of the KDF salt. Version 1 requires 16.
IV length	4 bytes	Length of the CBC IV. Version 1 requires 16.
Wrapped DEK length	4 bytes	Length of the encrypted DEK ciphertext. Version 1 requires 48.
MAC length	4 bytes	Length of the authentication tag. Version 1 requires 32.
KDF salt	16 bytes	Random salt used with the password to derive the KEK material.
IV	16 bytes	Random CBC initialization vector used to wrap the DEK.
Wrapped DEK	48 bytes	AES-256-CBC encrypted form of the 32-byte DEK, including padding.
Authentication tag	32 bytes	HMAC-SHA256 output computed over the header, salt, IV, and wrapped DEK.

For the initial implementation, the fixed-size header is 64 bytes. The full DEK+ object is expected to be 176 bytes:

Component:	Size
Header	64 bytes
KDF salt	16 bytes
IV	16 bytes
Wrapped DEK	48 bytes
HMAC-SHA256	32 bytes
Total	176 bytes

5.2.2 DEK+ Header Structure

```

/*****
 *
 * Structure: H5FD_dekplus_hdr_t
 *
 * Info:
 *
 *   Defines the in-memory representation of the fixed-size header associated
 *   with a DEK+ object. The fields in this structure describe the DEK+ format
 *   version, serialized object state, password/KDF parameters, wrapping cipher
 *   parameters, authentication parameters, and the lengths of the payload fields
 *   that follow the fixed header.
 *
 *   A DEK+ object may be embedded within the second page of an encrypted HDF5

```

* file. Support for storing this object in an external file is reserved for
 * future development. Consequently, this structure describes the DEK+ format
 * itself and does not imply a particular storage location.

* This structure does not contain the variable-length DEK+ payload. In the
 * normal password-protected initial implementation representation, the
 * serialized header is followed by the KDF salt, CBC initialization vector,
 * wrapped DEK ciphertext, and authentication tag. Alternate representations,
 * including the explicitly authorized cleartext DEK representation, are
 * interpreted according to the flags and field lengths stored in this header.

* This structure must not contain secret key material. The user password,
 * derived KEK material, plaintext or cleartext DEK, and temporary decrypted
 * buffers must not be stored in this structure.

* Serialization:

* This structure is an in-memory representation and must not be written
 * directly to persistent storage. Multi-byte integer fields are serialized
 * explicitly in little-endian byte order. Byte-array fields are serialized as
 * exact byte sequences. Implementations must not depend on compiler padding,
 * native structure layout, or host byte order.

* Fields:

* uint_8_t magic: Fixed DEK+ object identifier used to distinguish the DEK+
 * format from unrelated data and to reject obviously invalid input. The
 * expected initial implementation value is "EVFDKEY\0".

* uint8_t reserved[3]: Reserved bytes for future use. For the initial
 * implementation, these bytes should be set to zero and ignored on read
 * unless defined by a future format version. They are included to
 * preserve 32-bit alignment for the remaining header fields.

* uint8_t version: DEK+ format version. Included to allow for future
 * versions to add fields, change algorithms, or support additional
 * wrapped-key records while allowing older implementations to reject
 * unsupported formats safely. A value of 1 indicates the initial
 * implementation.

* uint32_t flags: Bitmask describing the serialized state and interpretation
 * of the DEK+ object. A value of zero indicates the normal, non-empty-
 * password representation. In the initial implementation, the values
 * include H5FD_DEKPLUS_FLAG_EMPTY_PASSWORD and
 * H5FD_DEKPLUS_FLAG_PLAINTEXT_DEK. These flags are derived by the
 * implementation from an authorized operation and are not intended to be
 * set directly by users. See section 5.2.3 for complete flag
 * definitions, mutual-exclusion rules, and validation requirements.

* uint32_t kdf_id: Identifier for the password-based key derivation function
 * used to derive KEK material from the user password and stored salt.
 * The initial KDF function selected is scrypt making the default value
 * of this field GCRY_KDF_SCRYPT. This algorithm was selected to make the
 * password-derived KEK recovery more resistant to brute-force attacks
 * than a simple hash or a lower-cost KDF. This field is zero when the

* cleartext-DEK representation does not use a KDF.

*
* uint32_t kdf_n: The scrypt CPU/memory parameter (N in the documentation).
* To satisfy the libgcrypt requirements, N must be greater than 1 and be
* a power of two. It is suggested to use a value of at least 2^{14} for
* interactive logins. As such, a value of $N = 32768$ will be used as the
* default for the initial implementation. This field is zero when the
* cleartext-DEK representation does not use a KDF.
*

* uint32_t kdf_r: The scrypt block size parameter (r in the documentation)
* scrypt block-size parameter r. A default size of 8 will be used for
* the initial implementation. This field is zero when the
* cleartext-DEK representation does not use a KDF.
*

* uint32_t kdf_p: The scrypt parallelization parameter (p in the
* documentation). To maintain simple and predictable derivation
* behavior, a default value of 1 will be used for the initial
* implementation.
*

* uint32_t kdf_out_len: The desired length of the derived key, in bytes,
* produced by the KDF (len in the documentation). For the initial
* implementation, a default value of 64 will be used. The first 32 bytes
* of the derived material will be used as the the AES-256 CBC wrapped
* key. The second 32 bytes are used as the HMAC-SHA256 authentication
* key (used for validation). This field is zero when the cleartext-DEK
* representation does not use a KDF.
*

* uint32_t cipher_id: Identifier for the cipher used to wrap the DEK. For the
* initial release, GCRY_CIPHER_AES256 will be used as the default
* cipher. Although only one cryptographic configuration is supported in
* the initial implementation, algorithm identifiers are included to
* facilitate future migration and version comparability without
* redesigning the file format. This field is zero when the cleartext-DEK
* representation does not use a wrapping cipher.
*

* uint32_t cipher_mode: Identifier for the cipher mode used to wrap the DEK.
* Necessary for utilizing algorithms with configurable modes. Since
* AES256 will be used to secure the DEK in the initial implementation,
* AES-256-CBC will be used as the default value in the initial
* implementation. Of note, CBC does not provide authentication; the
* wrapped DEK and associated DEK+ object content must be authenticated
* separately using an encrypt-then-MAC construction. This field is zero
* when the DEK is stored in cleartext.
*

* uint32_t mac_id: Identifier for the message authentication algorithm. When
* using an encryption algorithm and/or mode that does not provide
* authentication, an authentication algorithm must be specified. For the
* initial implementation, HMAC-SHA256 will be used as the default
* configuration. This selection provides integrity and tamper detection
* for the wrapped DEK and selected DEK+ metadata and is necessary
* when using AES256-CBC. This field is zero when no authentication
* algorithm is used.
*

* uint32_t dek_len: The length of the DEK represented by the DEK+ object.
* For the initial implementation, the Encryption VFD requires a decoded
* 32-byte AES256 key. As such, the default unwrapped DEK length is 32
* bytes.
*

```

*
*  uint32_t salt_len: The length of the KDF salt, in bytes, stored after the
*                    header. Salt is a non-secret value that must be stored with the key
*                    file and, when used with the user-provided password, is used derive
*                    the KEK. For the initial implementation, a default salt length of 16
*                    bytes will be used. This field is zero when no KDF salt is stored.
*
*  uint32_t iv_len: The length of the CBC initialization vector (IV), in bytes,
*                  stored after the salt. AES-256 in CBC mode, a fresh IV is required for
*                  each DEK wrapping operation. Since AES has a 16-byte block size, the
*                  default value for the IV length will be 16. This field is zero when
*                  the DEK is stored as cleartext.
*
*  uint32_t dek_payload_len: The length, in bytes, of the DEK payload. For the
*                           initial implementation, the wrapped DEK representation requires 48
*                           bytes of AES-256 ciphertext, including full-block padding. For a
*                           cleartext DEK representation, this field is expected to be 32 bytes.
*
*  uint32_t mac_len: The length, of the authentication tag, in bytes, stored
*                   after the wrapped DEK ciphertext. This is the full HMAC-SHA256 output
*                   used to authenticate the wrapped DEK and selected DEK+ metadata. This
*                   field is zero when no password-derived authentication tag is stored.
*
*****/

typedef struct H5FD_dekplus_hdr_t {
    uint8_t  magic[8];

    uint8_t  reserved[3];
    uint8_t  version;
    uint32_t flags;

    uint32_t kdf_id;
    uint32_t kdf_n;
    uint32_t kdf_r;
    uint32_t kdf_p;
    uint32_t kdf_out_len;

    uint32_t cipher_id;
    uint32_t cipher_mode;
    uint32_t mac_id;

    uint32_t dek_len;
    uint32_t salt_len;
    uint32_t iv_len;
    uint32_t dek_payload_len;
    uint32_t mac_len;
} H5FD_dekplus_hdr_t;

```

The `version` field is intentionally stored as a single byte. To preserve alignment, the three prior bytes are reserved for future use so that the remainder of the header remains aligned on 32-bit boundaries. For the initial implementation, the three reserved bytes should be written as zero.

5.2.3 DEK+ Header Flags

The DEK+ header flags describe the serialized state and interpretation of a DEK+ object. They are not intended to serve as direct user configuration settings. Users and applications instead select key management operations and runtime policies through the Encryption VFD configuration, the public key management API, or the h5keytools companion utility. When a permitted operation creates or rewrites a DEK+ object, the shared implementation derives and stores the appropriate header flags. When opening an existing DEK+ object, the implementation reads the flags and verifies that the encoded state is supported and permitted by the current runtime policy. The flags described in this section constitute the initially implemented DEK+ header flags set. As additional flags are deemed necessary, this section will be updated to reflect any revisions or additions. A `flags` value of zero represents the normal initial implementation password-protected form in which the DEK is wrapped and authenticated using KEK material derived from a non-empty password.

```
/** Empty Password Support */
/*
 * Indicates that the KEK is derived from the empty password ("").
 * Runtime will require `allow_empty_key_password` to be defined.
 */
#define H5FD_DEKPLUS_FLAG_EMPTY_PASSWORD (1u << 0)
```

This flag indicates the DEK is wrapped and authenticated using KEK material derived from a valid zero-length password string (""). The DEK+ object continues to contain a KDF salt, initialization vector, wrapped DEK, and authentication tag. This flag does not indicate that the password-based protection has been removed; it indicates that the password input used for KEK derivation is the empty string. This flag is set automatically when a DEK+ object is created or rekeyed using an empty-string password and the operation is authorized by the runtime policy. When opening the object, the implementation must reject empty-password recovery unless both this flag and the corresponding runtime authorization is enabled. When this flag is set, the KDF and MAC fields retain their expected non-zero values when processing a wrapped DEK. This flag should never be set when `H5FD_DEKPLUS_FLAG_CLEARTEXT_DEK` is set.

```
/** Cleartext DEK Support */
/*
 * Indicates no KEK exists (i.e., the wrapped_dek field stores a cleartext
 * DEK). The salt, IV, and MAC length fields are expected to be zero.
 * Runtime will require `allow_cleartext_dek` to be defined.
 */
#define H5FD_DEKPLUS_FLAG_CLEARTEXT_DEK (1u << 1)
```

This flag indicates that password-based protection has been removed and the DEK is stored directly within the DEK+ object without deriving a KEK. For this representation, password-derived KEK material, a KDF salt, an initialization vector, and a wrapped DEK ciphertext are not used. This flag is set automatically by an authorized Remove Password operation. It must not be set merely because

the user supplied an empty-string password. When opening the object, the implementation must reject cleartext DEK+ recovery unless both this flag and the corresponding runtime authorization is enabled. This flag should never be set when `H5FD_DEKPLUS_FLAG_EMPTY_PASSWORD` is set. When this flag is set, `kdf_id`, `kdf_n`, `kdf_r`, `kdf_p`, `kdf_out_len`, `cipher_id`, `cipher_mode`, `mac_id`, `salt_len`, `iv_len`, and `mac_len` are expected to be zero. Additionally, `dek_payload_len` is expected to be 32.

To validate the existing flags, the known bit mask should be defined and then validated. This may take the form of:

```
/* Define the known bit mask */
#define H5FD_DEKPLUS_FLAGS_ALL_KNOWN \
    (H5FD_DEKPLUS_FLAG_EMPTY_PASSWORD | \
     H5FD_DEKPLUS_FLAG_CLEARTEXT_DEK)

/* Validate the DEK+ Header Flags */
if (hdr->flags & ~H5FD_DEKPLUS_FLAGS_ALL_KNOWN)
    HGOTO_ERROR(..., "unsupported DEK+ header flag");
```

Unknown flags will cause rejection of the DEK+ object unless those bits are explicitly documented as something that may be ignored.

5.2.4 Serialization and Endianness

The initial version of the DEK+ format serializes all multi-byte integer fields into little-endian byte order. This applies to `flags`, algorithm identifiers, KDF parameters, and length fields. Byte-array fields such as `magic`, `reserved`, KDF salt, wrapped DEK, and authentication tag are written exactly as byte sequences and do not require endian conversion. Read/write helpers should explicitly encode and decode all fixed-width integer fields according to the DEK+ little-endian on-disk representation. Doing so avoids relying on host-native structure layout and keeps the DEK+ format portable across supported platforms.

5.2.5 Cryptographic Construction

The initial implementation uses an Encrypt-then-MAC construction. An Encrypt-then-MAC construction is a cryptographic design pattern in which the plaintext is first encrypted to produce ciphertext, after which a separate authentication tag is computed over the resulting ciphertext and associated metadata. During recovery, the authentication tag is verified before any decryption operation is attempted. This ordering ensures that corrupted or maliciously modified ciphertext is detected prior to decryption and is widely regarded as the preferred composition method when confidentiality and integrity are provided by separate cryptographic primitives.

The DEK is wrapped using AES-256-CBC. Both integrity and authenticity are provided through an HMAC-SHA256 authentication tag. The authentication tag is computed over the DEK+ header, KDF

salt, IV, and wrapped DEK prior to being written to disk. During recovery, the authentication tag is verified before the wrapped DEK is decrypted.

The KDF salt should be generated with secure random number generation when the DEK+ object is created. Salt is not considered secret or privileged information and must be stored in the DEK+ object as it is a necessary component for reconstructing the KEK from the user's password. The length of the KDF salt, in bytes, depends on the underlying cryptographic algorithm (for our purposes, 16 bytes is appropriate). Regarding the actual KDF function selection, both `PBKDF2` and `scrypt` are KDFs supported by `libgcrypt` as variable cost algorithms. Between these two options, `scrypt` provides stronger resistance to offline password-guessing attacks due to the amount of memory overhead required for each password attempt¹¹. This does raise the memory overhead of decrypting the DEK; however, this will on average be on the order of megabytes of memory and should not lead to bottlenecks or performance issues¹². The DEK is a 32-byte binary `AES-256` key. The KDF produces 64 bytes of KEK material using `scrypt` with $N = 32768$, $r = 8$, and $p = 1$. The first 32 bytes (the wrapping key) are used as the `AES-256-CBC` wrapping key, and the second 32 bytes (the authentication key) are used as the `HMAC-SHA256` key for encrypt-then-MAC authentication.

The Initialization Value (IV) used to encrypt the DEK should also be stored in the DEK+ object. Like the salt, the IV is not a secret value, but it does need to be unique for the encryption operation under the derived KEK. A new IV should be used for each individual encryption operation. The size of the IV depends on the encryption mode selected. Since `AES-256-CBC` is already utilized in the Encryption VFD, it will be used for the initial implementation, requiring the IV to be 16 bytes.

Although many binary file formats employ checksums to detect accidental corruption, a separate checksum is intentionally omitted from the DEK+ format. The `HMAC-SHA256` authentication tag already provides integrity verification for the authenticated contents of the file, detecting both accidental corruption and intentional modification. Since an attacker capable of modifying the DEK+ object can trivially recompute an unencrypted checksum but cannot recompute a valid `HMAC` without knowledge of the derived authentication key, the `HMAC` subsumes the role traditionally served by a checksum while providing substantially stronger security guarantees. In fact, the `HMAC` simultaneously provides integrity and authenticity.

¹¹ If an adversary obtains a copy of the DEK+ object, password-guessing attacks become the primary means of attempting to recover the wrapped DEK. Consequently, the strength of the user-defined password and the computational cost of the KDF directly influence the resistance of the solution of offline password-guessing attacks.

¹² Although `PBKDF2` remains a widely accepted and secure password-based KDF, it is primarily CPU-bound and generally requires a continually increasing iteration count to maintain resistance against attacks using modern hardware. By comparison, `scrypt` introduces a significant memory requirement in addition to the computational cost, making large-scale GPU- and ASIC-based password-guessing attacks considerably more expensive. It was for this reason that `scrypt` was selected as the default KDF for the initial implementation.

5.2.6 Secret and Non-Secret Material

The DEK+ format intentionally distinguishes between information that must remain confidential and information that is required only to interpret or validate the DEK+ object. This distinction influences both the on-disk representation and the application's memory-management strategy. The following values are not considered secret and may be stored directly within the DEK+ object and processed using ordinary application memory:

- DEK+ header
- KDF parameters
- KDF salt
- IV
- wrapped DEK
- authentication tag

Although the wrapped DEK contains confidential information in the cryptographic sense, it is already encrypted and therefore does not require protection by the `libgcrypt` secure memory pool. In contrast, the following values are considered secret material and should be allocated from the `libgcrypt` secure memory pool whenever practical:

- User password (while being processed)
- Derived KEK material
- Recovered plaintext DEK

The user-provided password and derived KEK only exist for the duration required to authenticate the DEK+ object and recover or rewrap the DEK and should be securely erased immediately after use. The recovered DEK remains resident in the secure memory pool for the lifetime of the associated encrypted HDF5 file (or Encryption VFD instance).

5.3 Code Components

To accommodate the local key encryption solution, the implementation should expose DEK+ operations through an `H5FDctl()`-style control interface rather than through many independent public functions. This will keep the public API aligned with the broader HDF5 VFD control model while allowing the Encryption VFD and `h5keytools` utility to share the same internal DEK+ implementation. This code will likely be defined a new H5FD submodule (likely `H5FDcrypt_utilities.*`).

The general control interface is expected to follow the pattern:

```
herr_t H5FDctl(
    H5FD_t *file,
    uint64_t op_code,
    uint64_t flags,
    const void *input,
    void **output
```

```
);
```

The `op_code` value identifies the requested key management operation. The `flags` argument provides VFD-specific control semantics defined by the broader `H5FDctl()` interface. DEK+-specific flags are expected to be passed as part of the input structure (i.e., they are expected to be present within `H5FD_dekplus_args_t::operation_flags`). The input pointer references an operation-specific input structure, and the output pointer may be used to return operation-specific output, such as a pointer to a recovered DEK in the `libgcrypt` secure memory pool.

5.3.1 Runtime Policy and Operation Flags

Runtime policy and operation flags control the behavior of an individual key management request. Unlike the flags stored in `H5FD_dekplus_hdr_t`, runtime flags are not serialized into the DEK+ object and do not describe its persistent cryptographic representation. Instead, they indicate which actions the current caller requests and which security-sensitive behaviors the current execution environment permits. The runtime controls are supplied through the `H5FD_dekplus_args_t` structure. Operation flags describe procedural behavior, such as whether terminal prompting is permitted, whether an existing DEK+ object should be overwritten, or whether a non-interactive caller has explicitly acknowledged a security-reducing operation. Runtime policy fields, such as `allow_empty_password` and `allow_cleartext_dek`, authorize specific cryptographic representations or transitions. A runtime authorization does not by itself modify the DEK+ header; the implementation derives the resulting serialized flags only after validating the requested operation and after successfully rewriting the DEK+ object.

Runtime controls must be validated independently from the serialized header flags. A serialized flag identifies the representation containing an existing DEK+ object, while the corresponding runtime authorization determines whether the current process is permitted to use or produce that representation. When both are required, the operation must fail unless the serialized state, runtime authorization, and build-time capabilities are mutually consistent.

Currently, several operation flags will be utilized. These flags are defined and discussed below.

```
/** Bypass Terminal-Based Password Prompt Support */
/*
 * Indicates the password prompt should be bypassed. Expected to be used with
 * `H5FD_DEKPLUS_FLAG_EMPTY_PASSWORD` or `H5FD_DEKPLUS_FLAG_CLEARTEXT_DEK`.
 * Runtime will require `allow_password_prompt_bypass` to be defined.
 */
#define H5FD_DEKPLUS_OP_FLAG_BYPASS_PASSWORD_PROMPT (1ULL << 1)
```

This runtime operation flag requires the key management workflow to proceed without displaying an interactive terminal-based password prompt. It is used for programmatic workflows in which all required password input has already been supplied through `H5FD_dekplus_args_t`, or when the DEK+ object does not require a non-empty password. This flag does not authorize empty-string password usage or cleartext DEK representation. Empty-string-password processing requires the applicable build-time capability, the `allow_empty_password` runtime authorization, and when

reading an existing object, the `H5FD_DEKPLUS_FLAG_EMPTY_PASSWORD` operation flag. Cleartext DEK processing similarly requires its applicable build-time capability, `allow_cleartext_dek`, and, when reading an existing DEK+ object, `H5FD_DEKPLUS_FLAG_CLEARTEXT_DEK`.

The implementation must reject the operation if bypassing the prompt would leave the required authentication input unavailable. For example, a password-protected DEK+ object cannot be unlocked with this flag set unless a valid password is supplied programmatically. This flag only affects terminal-based password collection. It does not suppress mandatory security warnings or act as a substitute for the explicit acknowledgment of a security-reducing operation.

```
#define H5FD_DEKPLUS_OP_FLAG_CONFIRM_SECURITY_REDUCTION (1ULL << 2)
```

This runtime operation flag records the caller's explicit acknowledgement that the requested operation reduces the protection applied to the DEK. It is required for non-interactive operations that remove password-based protection, creation or retention of a cleartext DEK representation, the exportation of a plaintext DEK, or otherwise transition of key material from a more protected representation into a less protected representation. For interactive workflows, the implementation should obtain equivalent confirmation directly from the user after displaying a clear description of the security consequences. The implementation may then set this flag before invoking the underlying shared implementation. For non-interactive workflows, this flag must be supplied explicitly by the calling application or derived from an equally explicit API argument.

This flag records acknowledgment. It does not enable the underlying capabilities. The requested functionality must also be present in the build, permitted by the runtime policy fields, and be valid for the selected operation code. Setting this flag alone must never permit password removal, cleartext storage, or DEK exportability. However, this flag is not required for operations that preserve or strengthen the existing security model of a DEK or DEK+ object. For example, transferring from one non-empty password to another, adding password protection to a cleartext DEK+ object, or increasing the KDF work factor.

```
/** External DEK+ File Support (For Future Development) */
/*
 *   Intended for future use. Will indicate that the DEK+ format content should
 *   be saved to an external file.
 */
#define H5FD_DEKPLUS_OP_FLAG_EXTERNAL_COPY (1ULL << 5)
```

This runtime operation flag requests that the operation create a complete external DEK+ object in addition to, or instead of, updating the embedded DEK+ object associated with the encrypted HDF5 file. The external object may be used as a backup, recovery copy, or independently password-

protected duplicate of the same underlying DEK. The external DEK+ object is a complete, serialized representation rather than a reference to the embedded DEK+ object. It therefore contains its own header, KDF parameters, KDF salt, initialization vector, DEK payload, and authentication tag. When the external copy is protected with a different password, the implementation must recover the existing DEK into the `libgcrypt` secure memory pool and create new KDF salt, IV, KEK material, wrapped DEK, and authentication tag for the external destination. The encrypted HDF5 data and underlying DEK remain unchanged.

The caller must provide a valid `destination_path`. If the destination already exists, replacement is permitted only when `allow_overwrite` is true. The `H5FD_DEKPLUS_OP_FLAG_ATOMIC_REPLACE` operation flag may additionally be supplied to require atomic replacement of an existing external object. This operation flag does not alter `H5FD_dekplus_hdr_t::flags`; the DEK+ object storage location does not alter the cryptographic interpretation of the underlying DEK+ object. It also does not authorize password removal, cleartext DEK storage, or plaintext DEK export. Any external copy must independently satisfy the build-time and runtime policies applicable to its selected representation.

To validate the existing operational flags, the known bit mask should be defined and then validated. This may take the form of:

```
/* Define the operational flags bit mask */
#define H5FD_DEKPLUS_OP_FLAG_ALL_KNOWN \
    H5FD_DEKPLUS_OP_FLAG_BYPASS_PASSWORD_PROMPT | \
    H5FD_DEKPLUS_OP_FLAG_CONFIRM_SECURITY_REDUCTION | \
    H5FD_DEKPLUS_OP_FLAG_ATOMIC_REPLACE | \
    H5FD_DEKPLUS_OP_FLAG_RETURN_DEK | \
    H5FD_DEKPLUS_OP_FLAG_CREATE_EXTERNAL_COPY)

/* Validate the operational flags */
if (args->operation_flags & ~H5FD_DEKPLUS_OP_FLAG_ALL_KNOWN)
    HGOTO_ERROR(..., "unsupported DEK+ operation flag");
```

5.3.2 Public Control Operations (H5FDctl)

The public DEK+ operations should be represented as `H5FDctl` operation codes. These operation codes should mirror the supported operations described in Section 2.5 so that the Encryption VFD and `h5keytools` use the same conceptual operation model. The initial operation set should include:

```
#define H5FD_CTL_CRYPT_UTILS_CREATE /* Create DEK+ object */
#define H5FD_CTL_CRYPT_UTILS_DEKPLUS_UNLOCK /* Recover DEK from DEK+ object */
#define H5FD_CTL_CRYPT_UTILS_DEKPLUS_VALIDATE /* Validate DEK+ object */
#define H5FD_CTL_CRYPT_UTILS_DEKPLUS_CREATE_DEK /* Generate cleartext DEK */
```

```
#define H5FD_CTL_CRYPT_UTILS_DEKPLUS_CHANGE_PASSWORD /* Change password / rotate
KEK */

#define H5FD_CTL_CRYPT_UTILS_DEKPLUS_REMOVE_PASSWORD /* Convert to cleartext DEK
representation */

#define H5FD_CTL_CRYPT_UTILS_DEKPLUS_ADD_PASSWORD /* Convert from cleartext DEK
representation */

#define H5FD_CTL_CRYPT_UTILS_DEKPLUS_UPDATE_KDF /* Rewrap using updated KDF
parameters */

#define H5FD_CTL_CRYPT_UTILS_DEKPLUS_MIGRATE /* Migrate to future DEK+ format */

#define H5FD_CTL_CRYPT_UTILS_EXPORT_DEK /* Export DEK to cleartext mechanism */

#define H5FD_CTL_CRYPT_UTILS_IMPORT_DEK /* Import cleartext DEK into DEK+ */
```

For normal Encryption VFD operation, the two primary control operations are:

```
H5FDctl(file, H5FD_CTL_CRYPT_UTILS_DEKPLUS_CREATE, flags, input, output);
H5FDctl(file, H5FD_CTL_CRYPT_UTILS_DEKPLUS_UNLOCK, flags, input, output);
```

H5FD_CTL_CRYPT_UTILS_DEKPLUS_CREATE is used when a new encrypted HDF5 file requires the creation of a DEK+ object or embedded DEK+ object. H5FD_CTL_CRYPT_UTILS_DEKPLUS_UNLOCK is used when opening an existing encrypted HDF5 file that is already associated with a DEK+ object. Other operations, such as password changes, password removal, KDF updates, format migration, import, and export, are expected to be invoked primarily through the `h5keytools` utility. These operations should still use the same underlying control operation model and internal helper functions, but they should not be part of the normal Encryption VFD open path.

5.3.3 Internal High-Level DEK+ Operations

The internal DEK+ operations provide the primary implementation entry points used by both the Encryption VFD and the `h5keytools` companion utility. Unlike the lower-level helper routines described in later sections, these functions orchestrate complete key management workflows by coordinating header initialization, password validation, cryptographic processing, secure-memory allocation, and DEK+ serialization. Each operation is responsible for invoking the appropriate internal helper functions while maintaining consistent error handling and cleanup semantics.

```
herr_t H5FD__crypt_utils_dekplus_create(const char *source_path,
bool allow_empty_key_password, bool allow_existing_key, const char
*password, uint8_t **out_secure_dek);
```

Creates a new DEK+ object. When `allow_existing_key` is false, a new 32-byte DEK is generated using the `libgcrypt` random-number generator and is allocated from the `libgcrypt` secure memory pool. Otherwise, the user-supplied DEK is copied into the secure memory pool and used directly. The supplied password is validated against the current password policy, 64 bytes of KEK material are

derived using the configured Key Derivation Function, the DEK is wrapped using AES-256-CBC, an HMAC-SHA256 authentication tag is computed, and the resulting DEK+ object is written to persistent storage. Upon successful completion, the function returns a pointer to the recovered plaintext DEK residing within the `libgcrypt` secure memory pool.

Support for operation with an empty password should be permitted only when explicitly requested through the `allow_empty_key_password` flag, and only when working on a trusted, isolated system or in a non-interactive execution environment where a password prompt is impractical. Because an empty password effectively bypasses password-based protection of the wrapped DEK, DEK+ objects created in this manner should not be considered secure. They may subsequently be rekeyed using `h5keytools` to restore password-based protection. This routine is primarily used when creating a new encrypted HDF5 file.

```
herr_t      H5FD_dekplus_unlock(const      char      *source_path,      bool
allow_empty_key_password,      const      char      *password,      uint8_t
**out_secure_dek);
```

Reads the fixed header and payload from the configured DEK+ storage location, validates the header, reconstructs the KEK material using the configured Key Derivation Function, verifies the HMAC-SHA256 authentication tag, decrypts the wrapped DEK, allocates secure memory for the recovered plaintext DEK, and returns a pointer to the recovered 32-byte binary DEK stored in the `libgcrypt` secure memory pool. The user-provided password and KEK material are securely erased before returning to the caller. Primarily used when opening an existing encrypted HDF5 file.

5.3.4 Omnibus Input and Output Structures

Each `H5FDctl` operation should define a single omnibus input structure rather than passing function parameters independently. This keeps the public interface stable while allowing future fields to be added without changing the control function signature. Each field in this structure will explicitly state which functions require a non-zero value.

```
/*
 *
 * Structure: H5FD_dekplus_args_t
 *
 * Purpose:
 *
 * Provides the omnibus input arguments used by public DEK+ control
 * operations. The same structure is supplied to all DEK+ H5FDctl()
 * operations so that the public control interface remains stable as new
 * key management operations and optional parameters are introduced.
 *
 * Fields that are not applicable to the selected operation must be set to
 * zero, false, or NULL as appropriate. The implementation must validate
 * all required fields before performing cryptographic processing and must
 * reject contradictory, unsupported, or unexpectedly populated fields
 * when their presence could create ambiguity.
 */
```

```

*
* Password pointers refer to null-terminated strings. A valid pointer to a
* zero-length string represents an empty-string password. A NULL password
* pointer means that no password was supplied programmatically and may
* cause an interactive operation to prompt the user. NULL must not be
* interpreted as an empty-string password.
*
* Secret input fields, including passwords and existing_dek, are owned by
* the caller. The implementation must copy secret material into the
* libgcrypt secure memory pool when retention is required and must not
* modify or free caller-owned buffers.
*
* Fields:
*
*   size_t struct_size:
*       Size, in bytes, of the caller-provided structure. Must be set to
*       sizeof(H5FD_dekplus_args_t) for every operation. Used to support
*       future extension and reject incompatible structures.
*
*   Non-zero for:
*       All DEK+ control operations.
*
*   uint32_t struct_version:
*       Version of the public argument structure. The initial value is 1.
*
*   Non-zero for:
*       All DEK+ control operations.
*
*   const char *source_path:
*       Path to the existing object or containing HDF5 file from which DEK
*       material is read.
*
*   Non-NULL for:
*       UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
*       ADD_PASSWORD, UPDATE_KDF, MIGRATE, EXPORT_DEK.
*
*   Optional for:
*       IMPORT_DEK when the input DEK is supplied through existing_dek.
*
*   NULL for:
*       CREATE_DEK unless an implementation writes the generated DEK
*       directly to a requested path.
*
*   const char *destination_path:
*       Path to the destination object or containing HDF5 file that will be
*       created or rewritten.
*
*   Non-NULL for:
*       CREATE, MIGRATE, EXPORT_DEK, IMPORT_DEK.
*
*   Optional for:
*       CHANGE_PASSWORD, REMOVE_PASSWORD, ADD_PASSWORD, UPDATE_KDF when
*       the source object is rewritten in place.
*
*   NULL for:
*       UNLOCK and validation-only operations.

```

```

* uint64_t operation_flags:
*   Runtime operation flags controlling behavior such as interactive
*   prompting, overwrite authorization, atomic replacement, or explicit
*   confirmation of a security-reducing operation. These flags are not
*   serialized as H5FD_dekplus_hdr_t::flags unless the DEK+ operation
*   independently derives a corresponding on-disk state.
*
*   May be non-zero for:
*       All operations.
*
* const uint8_t *existing_dek:
*   Caller-provided plaintext DEK to import or use instead of generating
*   a new random DEK.
*
*   Non-NULL for:
*       CREATE when an existing DEK is supplied;
*       IMPORT_DEK when the DEK is supplied directly in memory.
*
*   NULL for:
*       UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
*       ADD_PASSWORD, UPDATE_KDF, and ordinary MIGRATE operations.
*
* size_t existing_dek_len:
*   Length, in bytes, of existing_dek. Version 1 requires 32 whenever
*   existing_dek is non-NULL.
*
*   Non-zero for:
*       CREATE or IMPORT_DEK when existing_dek is non-NULL.
*
*   Zero for:
*       All other operations.
*
* const char *password:
*   Current or initial password used to create, authenticate, or recover
*   a DEK+ object.
*
*   Non-NULL for non-interactive:
*       CREATE, UNLOCK, CHANGE_PASSWORD, REMOVE_PASSWORD, UPDATE_KDF,
*       MIGRATE, EXPORT_DEK, and authenticated VALIDATE.
*
*   Optional for:
*       Interactive forms of those operations, where NULL causes the
*       implementation or h5keytools to prompt the user.
*
*   Normally NULL for:
*       CREATE_DEK and ADD_PASSWORD when the source DEK+ object contains
*       a plaintext DEK and no current password is required.
*
* const char *new_password:
*   Replacement password used to protect an existing DEK.
*
*   Non-NULL for non-interactive:
*       CHANGE_PASSWORD and ADD_PASSWORD.
*
*   Optional for:
*       Interactive CHANGE_PASSWORD and ADD_PASSWORD, where NULL causes
*       the user to be prompted.

```

```
*
*
*     NULL for:
*         CREATE, UNLOCK, VALIDATE, REMOVE_PASSWORD, CREATE_DEK,
*         EXPORT_DEK, IMPORT_DEK, and operations that do not establish a
*         new password.
*
* bool allow_empty_password:
*     Runtime authorization permitting creation or rekeying with a valid
*     empty-string password. This does not request password removal and
*     does not authorize plaintext DEK storage.
*
*     True only for:
*         CREATE, CHANGE_PASSWORD, or ADD_PASSWORD when an empty-string
*         password is intentionally requested;
*         UNLOCK or authenticated VALIDATE when opening an object carrying
*         H5FD_DEKPLUS_FLAG_EMPTY_PASSWORD.
*
*     False for:
*         Ordinary non-empty-password operations and REMOVE_PASSWORD.
*
* bool allow_cleartext_dek:
*     Runtime authorization permitting creation, opening, or rewriting of
*     a DEK+ object carrying H5FD_DEKPLUS_FLAG_PLAINTEXT_DEK.
*
*     True only for:
*         REMOVE_PASSWORD;
*         ADD_PASSWORD when the source object stores a plaintext DEK;
*         UNLOCK, VALIDATE, MIGRATE, or EXPORT_DEK when the source or
*         destination representation requires plaintext-DEK support.
*
*     False for:
*         Normal password-protected and empty-string-password operations.
*
* bool allow_existing_dek:
*     Indicates that CREATE may use existing_dek rather than generating a
*     new random DEK.
*
*     True only for:
*         CREATE when existing_dek and existing_dek_len are valid.
*
*     False for:
*         All other operations.
*
* bool allow_overwrite:
*     Explicitly permits replacement of an existing destination object.
*
*     May be true for:
*         CREATE, CHANGE_PASSWORD, REMOVE_PASSWORD, ADD_PASSWORD,
*         UPDATE_KDF, MIGRATE, EXPORT_DEK, and IMPORT_DEK.
*
*     False for:
*         UNLOCK, VALIDATE, and CREATE_DEK operations that do not write to
*         an existing destination.
*
* uint32_t source_format:
*     Identifier for the input DEK handling mechanism.
*
```

```

*      Non-zero for:
*          MIGRATE, IMPORT_DEK, and EXPORT_DEK when the source mechanism
*          cannot be inferred unambiguously.
*
*      Zero for:
*          Operations that operate only on a known DEK+ representation.
*
*      uint32_t target_format:
*          Identifier for the requested output DEK handling mechanism.
*
*      Non-zero for:
*          MIGRATE, IMPORT_DEK, and EXPORT_DEK.
*
*      Zero for:
*          CREATE, UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
*          ADD_PASSWORD, and UPDATE_KDF when the target remains DEK+.
*
*      uint32_t kdf_id:
*          Optional override for the KDF identifier used when writing a new
*          wrapped-DEK representation.
*
*      May be non-zero for:
*          CREATE, CHANGE_PASSWORD, ADD_PASSWORD, UPDATE_KDF, MIGRATE,
*          and IMPORT_DEK.
*
*      Zero means:
*          Use the Version 1 default or retain the existing value.
*
*      uint32_t kdf_n:
*          Optional scrypt N override.
*
*      May be non-zero for:
*          CREATE, CHANGE_PASSWORD, ADD_PASSWORD, UPDATE_KDF, MIGRATE,
*          and IMPORT_DEK.
*
*      Zero means:
*          Use the default or retain the existing value.
*
*      uint32_t kdf_r:
*          Optional scrypt r override. Operation applicability is identical to
*          kdf_n. Zero selects the default or existing value.
*
*      uint32_t kdf_p:
*          Optional scrypt p override. Operation applicability is identical to
*          kdf_n. Zero selects the default or existing value.
*
*      uint32_t kdf_out_len:
*          Optional derived-material length override. Version 1 normally uses
*          64 and should reject unsupported values.
*
*      May be non-zero for:
*          CREATE, CHANGE_PASSWORD, ADD_PASSWORD, UPDATE_KDF, MIGRATE,
*          and IMPORT_DEK.
*
*      Zero means:
*          Use the Version 1 default or retain the existing value.
*

```

```

/*****
typedef struct H5FD_dekplus_args_t {
    size_t    struct_size;
    uint32_t  struct_version;

    const char *source_path;
    const char *destination_path;

    uint64_t operation_flags;

    const uint8_t *existing_dek;
    size_t        existing_dek_len;

    const char *password;
    const char *new_password;

    bool allow_empty_password;
    bool allow_cleartext_dek;
    bool allow_existing_dek;
    bool allow_overwrite;

    uint32_t source_format;
    uint32_t target_format;

    uint32_t kdf_id;
    uint32_t kdf_n;
    uint32_t kdf_r;
    uint32_t kdf_p;
    uint32_t kdf_out_len;
} H5FD_dekplus_args_t;

```

Similarly, DEK+ operations should return a single omnibus structure rather than singular output.

```

/*****
 *
 * Structure: H5FD_dekplus_result_t
 *
 * Purpose:
 *
 *     Provides the omnibus result returned by DEK+ H5FDctl() operations. The
 *     same result structure is used for creation, recovery, validation,
 *     password-management, migration, import, and export operations.
 *
 *     Fields that are not applicable to the completed operation are returned
 *     as zero, false, or NULL. A successful H5FDctl() return value indicates
 *     that the requested operation completed successfully; the validation
 *     fields provide additional detail for operations that distinguish
 *     structural validation from authentication validation.
 *
 *     When secure_dek is non-NULL, it points to secret material allocated from
 *     the libgcrypt secure memory pool. Ownership is transferred to the caller
 *     unless the calling interface explicitly documents otherwise. The caller
 *     must release the allocation using the corresponding libgcrypt memory
 *     management routine and must not release it with free().

```

```

*
* Fields:
*
*   size_t struct_size:
*       Size, in bytes, of the result structure populated by the
*       implementation. Set to sizeof(H5FD_dekplus_result_t) for every
*       successfully initialized result.
*
*       Non-zero for:
*           All operations returning this structure.
*
*   uint32_t struct_version:
*       Version of the public result structure. The initial value is 1.
*
*       Non-zero for:
*           All operations returning this structure.
*
*   uint8_t *secure_dek:
*       Pointer to the recovered, generated, or imported plaintext DEK stored
*       in the libgcrypt secure memory pool.
*
*       Non-NULL for:
*           CREATE, UNLOCK, CREATE_DEK, IMPORT_DEK, and operations explicitly
*           requested to return the recovered DEK.
*
*       Optional for:
*           CHANGE_PASSWORD, REMOVE_PASSWORD, ADD_PASSWORD, UPDATE_KDF,
*           MIGRATE, and EXPORT_DEK.
*
*       NULL for:
*           Structural validation that does not recover the DEK.
*
*   size_t secure_dek_len:
*       Length, in bytes, of secure_dek. Version 1 returns 32 whenever
*       secure_dek is non-NULL.
*
*       Non-zero when:
*           secure_dek is non-NULL.
*
*       Zero when:
*           secure_dek is NULL.
*
*   uint32_t detected_flags:
*       DEK+ header flags read from the source object or written to the
*       resulting destination object.
*
*       Meaningful for:
*           CREATE, UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
*           ADD_PASSWORD, UPDATE_KDF, MIGRATE, IMPORT_DEK, and EXPORT_DEK
*           when a DEK+ object is inspected or produced.
*
*       Zero for:
*           CREATE_DEK and formats without DEK+ header flags.
*
*   uint32_t detected_version:
*       DEK+ format version read from or written to the relevant object.
*

```

```

*      Non-zero for:
*          Operations that inspect or produce a DEK+ object.
*
*      Zero for:
*          CREATE_DEK and non-DEK+ representations.
*
*      uint32_t resulting_format:
*          Identifier for the DEK handling mechanism produced or detected by
*          the completed operation.
*
*      Non-zero for:
*          MIGRATE, IMPORT_DEK, EXPORT_DEK, and operations where the format
*          cannot otherwise be inferred.
*
*      May be zero for:
*          Operations that are unambiguously DEK+-specific.
*
*      bool structural_validation_performed:
*          Indicates whether the implementation examined the DEK+ header,
*          version, flags, algorithm identifiers, lengths, and total serialized
*          size.
*
*      True for:
*          VALIDATE, UNLOCK, password-management operations, UPDATE_KDF,
*          MIGRATE, and other operations that read an existing DEK+ object.
*
*      bool structural_validation_passed:
*          Indicates that structural validation completed successfully.
*          Meaningful only when structural_validation_performed is true.
*
*      bool authentication_performed:
*          Indicates whether the operation derived or otherwise obtained the
*          authentication key and verified the DEK+ authentication tag.
*
*      True for:
*          UNLOCK, authenticated VALIDATE, CHANGE_PASSWORD,
*          REMOVE_PASSWORD, UPDATE_KDF, MIGRATE, EXPORT_DEK, and other
*          operations that recover a wrapped DEK.
*
*      False for:
*          Structural-only validation and plaintext-DEK representations
*          that do not use password-derived authentication.
*
*      bool authentication_passed:
*          Indicates that authentication completed successfully. Meaningful
*          only when authentication_performed is true.
*
*      bool migration_required:
*          Indicates that the source object is structurally valid but requires
*          migration before the current implementation can use or rewrite it
*          in the requested form.
*
*      May be true for:
*          VALIDATE, UNLOCK, UPDATE_KDF, and MIGRATE.
*
*      False for:
*          Newly created Version 1 objects and operations unrelated to

```

```

*           format compatibility.
*
*****/
typedef struct H5FD_dekplus_result_t {
    size_t    struct_size;
    uint32_t  struct_version;

    uint8_t   *secure_dek;
    size_t    secure_dek_len;

    uint32_t  detected_flags;
    uint32_t  detected_version;
    uint32_t  resulting_format;

    bool structural_validation_performed;
    bool structural_validation_passed;
    bool authentication_performed;
    bool authentication_passed;
    bool migration_required;
} H5FD_dekplus_result_t;

```

5.3.5 Internal DEK+ Read/Write Helpers

The read/write helper functions should be implemented as internal routines and should not be exposed as public API entry points. These functions provide the interface between the DEK+ implementation and the serialized on-disk DEK+ representation. Several of these helper functions are defined below.

```

herr_t H5FD__crypt_utils_dekplus_read(const char *source_path, const
H5FD_dekplus_hdr_t *dekplus_hdr, const uint8_t *salt, const uint8_t
*iv, const uint8_t *wrapped_dek, const uint8_t *mac);

```

Reads the fixed header and payload from the configured DEK+ storage location. Primarily used when working with an existing HDF5 file that requires the Encryption VFD for operations.

```

herr_t H5FD__crypt_utils_dekplus_write(const char *source_path,
const H5FD_dekplus_hdr_t *dekplus_hdr, const uint8_t *salt, const
uint8_t *iv, const uint8_t *wrapped_dek, const uint8_t *mac);

```

Writes the fixed header and payload to the configured DEK+ storage location. Primarily used during initial DEK+ format creation, KEK rekey operations, or with future key format migration.

```

herr_t H5FD__crypt_utils_dekplus_init_header(H5FD_dekplus_hdr_t
*dekplus_hdr, uint32_t flags);

```

Creates a new key header with the default values:

```

magic           = "EVFDKEY\0"
version         = 1
kdf_id          = GCRY_KDF_SCRIPT
kdf_n           = 32768

```

```

kdf_r          = 8
kdf_p          = 1
kdf_out_len    = 64
cipher_id      = GCRY_CIPHER_AES256
cipher_mode    = GCRY_CIPHER_MODE_CBC
mac_id         = HMAC-SHA256 project identifier
dek_len        = 32
salt_len       = 16
iv_len         = 16
dek_payload_len = 48
mac_len        = 32

```

```

herr_t H5FD__crypt_utils_dekplus_validate_header(const
H5FD_dekplus_hdr_t *dekplus_hdr);

```

Function used to validate DEK+ object metadata before cryptographic use. Rejects malformed or unsupported DEK+ objects. Validation includes the following:

```

magic
version
flags
KDF parameters
cipher/mode/MAC identifiers
field lengths
expected total file size

```

These helper functions are responsible for reading and writing the fixed-size header payload fields, encoding and decoding little-endian integer fields, validating field lengths, checking supported format versions, and ensuring that the DEK+ object conforms to the expected Version 1 layout. Because these routines operate on the DEK+ serialized representation, they should process the DEK+ header, salt, IV, wrapped DEK, and authentication tag using ordinary application memory. These values are either metadata or already-encrypted or already authenticated data and do not require allocation from the `libgcrypt` secure memory pool.

5.3.6 Internal Cryptographic Helpers

The cryptographic helper functions encapsulate the primitive cryptographic operations required by the local key-generation solution. These routines are responsible for generating cryptographically secure random DEKs, deriving KEK material from user passwords through the selected Key Derivation Function, wrapping and unwrapping the DEK, and computing and verifying the authentication tag associated with the DEK+ object. Isolating these operations behind a well-defined interface simplifies maintenance of the Encryption VFD while ensuring that cryptographic operations remain centralized, consistent, and easier to audit. As the implementation evolves, these helpers also provide a natural location for introducing additional algorithms or cryptographic policies without requiring changes to the higher-level workflow. Representative internal helpers include:

```
herr_t H5FD__crypt_utils_generate_dek(uint8_t **out_secure_dek,
size_t dek_len);
```

Allocates `dek_len` bytes from `libgcrypt` secure memory pool and fills it with random bytes (32 bytes is the assumed size for the initial implementation). Memory allocation should be done using `gcry_malloc_secure(32)` and the random bytes should be generated with `gcry_randomize(..., GCRY_STRONG_RANDOM)`.

```
H5FD__crypt_utils_generate_dekplus_material(uint8_t salt[16],
uint8_t iv[16]);
```

Generates the random cryptographic material required when creating or rewriting a DEK+ object. Specifically, this helper generates a new KDF salt and CBC initialization vector using `libgcrypt`'s cryptographically secure random number generator. The generated values are non-secret and therefore do not require allocation from the `libgcrypt` secure memory pool.

```
herr_t H5FD__crypt_utils_derive_kek_material(const char *password,
const uint8_t *salt, size_t salt_len, const H5FD_dekplus_hdr_t
*dekplus_hdr, uint8_t **out_secure_kek);
```

Uses `scrypt` to derive 64 bytes. Of note, `kek_material[0..31]` is the AES-256-CBC wrapping key and `kek_material[32..63]` is the HMAC-SHA256 key.

```
herr_t H5FD__crypt_utils_wrap_dek(const uint8_t *secure_dek, const
uint8_t kek_material[64], const uint8_t iv[16], uint8_t
wrapped_dek[48]);
```

Encrypts the 32-byte binary DEK using AES-256-CBC with one block of padding.

```
herr_t H5FD__crypt_utils_recover_dek(const uint8_t wrapped_dek[48],
const uint8_t kek_material[64], const uint8_t iv[16], uint8_t
**out_secure_dek);
```

Decrypts and un pads the wrapped DEK. The result must be exactly 32 bytes and should be copied into `libgcrypt` secure memory.

```
herr_t H5FD__crypt_utils_compute_dekplus_mac(const
H5FD_dekplus_hdr_t *dekplus_hdr, const uint8_t *salt, const uint8_t
*iv, const uint8_t *wrapped_dek, uint8_t mac_out[32], const uint8_t
kek_material[64]);
```

Computes the HMAC-SHA256 over the authenticated DEK+ object contents.

```
herr_t H5FD__crypt_utils_verify_dekplus_mac(const H5FD_dekplus_hdr_t
*dekplus_hdr, const uint8_t *salt, const uint8_t *iv, uint8_t
*wrapped_dek, uint8_t expected_mac[32], const uint8_t
kek_material[64]);
```

Recomputes and compares the MAC using a constant-time comparison.

5.3.7 Password and Policy Helpers

The password and policy helper functions implement the runtime security policies associated with password handling and user interaction. These routines validate whether a particular operation is permitted under the current Encryption VFD configuration, determine whether empty-password operation is allowed, and populate the appropriate DEK+ format flags during key creation or KEK rotation. By separating policy enforcement from both the cryptographic operations and the file I/O, these helper functions ensure that password-related decisions remain consistent throughout the implementation and may be modified independently as additional authentication methods or configuration options are introduced.

```
herr_t H5FD__crypt_utils_validate_password_policy(bool is_create,
bool rekey_kek, bool allow_empty_key_password, const char *password,
uint32_t *out_flags);
```

Enforces the empty-password policy. The rules are as follows:

- On Creation:
 - Empty password is allowed only when `allow_empty_key_password == true`
- On Rekey:
 - Empty new password is allowed only when `rekey_kek == true && allow_empty_key_password == true`
- On Unlock:
 - Empty password is allowed only when the DEK+ object has `EMPTY_PASSWORD` flag present
 - If `CLEARTEXT_DEK` is present, password validation is skipped entirely.

```
bool H5FD__crypt_utils_password_is_empty(const char *password);
```

Determines whether the supplied password should be treated as an empty-string password. A `NULL` pointer is treated as an invalid programming error and is not considered equivalent to an empty password. Returns `true` only when the supplied password points to a valid zero-length C string (`""`).

```
void H5FD__crypt_utils_secure_clear(void (buffer, size_t size);
```

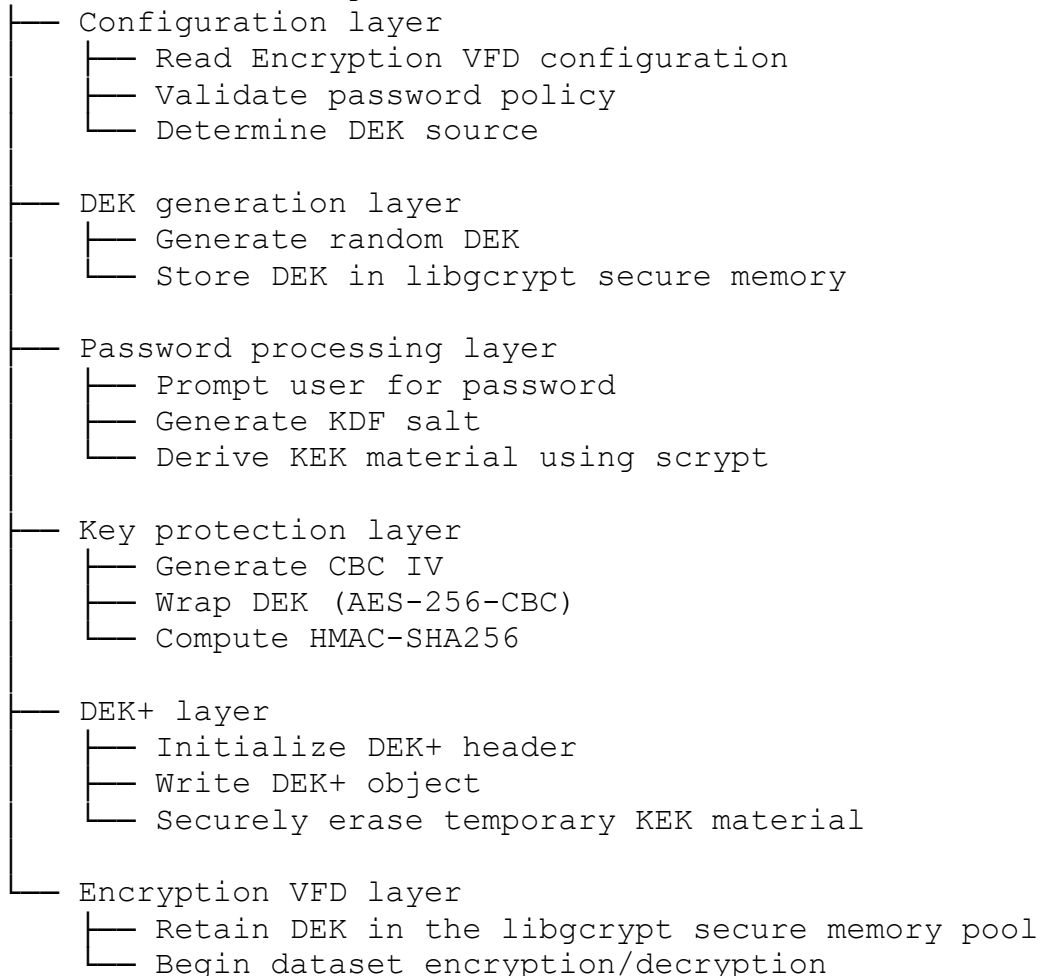
Securely overwrites sensitive memory before it is released or reused. This helper should use an implementation that prevents compiler optimization from removing the memory-clearing operation. It should be used for plaintext password buffers, derived KEK material, temporary plaintext DEK buffers, and any other temporary secret material that exists outside the `libgcrypt` secure memory pool.

Note: This helper should not be used to release `libgcrypt` secure memory allocations. Secure memory allocated through `libgcrypt` should instead be released using corresponding `libgcrypt` memory management routines.

5.4 Call-Stack Outlines

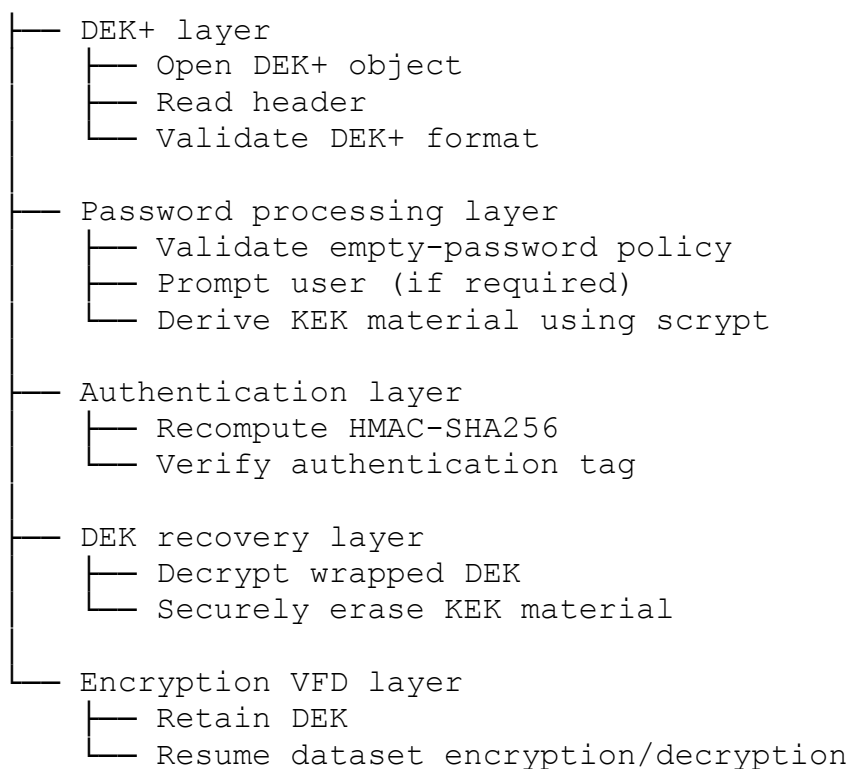
5.4.1 Creating a New DEK+ Object

Create New DEK+ Object



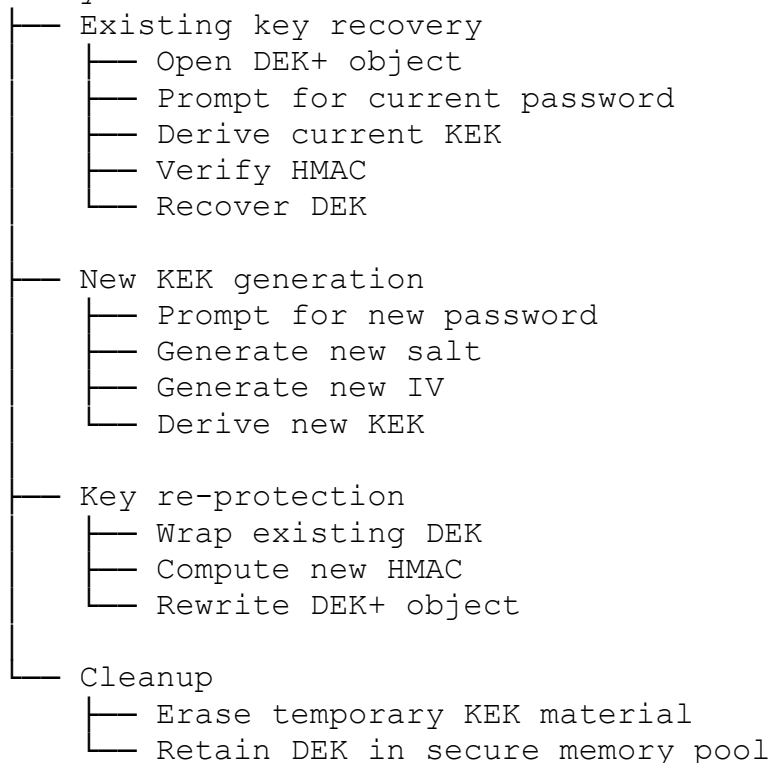
5.4.2 Unlock Existing DEK+ Object

Unlock Existing DEK+ Object



5.5 Rekey Workflow (h5keytools) (Password Change/Removal)

Rekey KEK



5.6 Testing Plan

As of the time of writing, the following validation scenarios will need to be validated with testing:

- Successful creation of a new DEK+ object.
- Successful recovery of a wrapped DEK.
- Incorrect password.
- Empty-password operation.
- Password-to-password KEK rotation.
- Password-to-empty KEK rotation.
- Empty-to-password KEK rotation.
- Corrupted header.
- Corrupted wrapped DEK.
- Corrupted HMAC.
- Invalid version.
- Invalid KDF parameters.
- Invalid cipher identifiers.
- Missing or inaccessible DEK+ object.
- Secure memory pool cleanup following both success and failure paths.

Appendix A – Glossary

- **Authentication Key:** The second 32 bytes of KEK material derived by the Key Derivation Function. Used exclusively by HMAC-SHA256 to compute and verify the DEK+ authentication tag.
- **Authentication Tag:** A value computed over protected data using a keyed authentication algorithm. In the DEK+ format, the authentication tag is the HMAC-SHA256 output computed over the header, KDF salt, IV, and wrapped DEK. It is used to detect accidental corruption and unauthorized modification before the wrapped DEK is decrypted.
- **Authenticity:** Assurance that the authenticated portions of the DEK+ format were produced or modified only by an entity possessing the correct authentication key derived from the user password.
- **Binary blob:** An opaque sequence of bytes interpreted by the application rather than a standardized file format.

- **Cipher:** A cryptographic algorithm used to transform plaintext into ciphertext, or ciphertext back into plaintext, using a key. In the initial DEK+ implementation, AES-256-CBC is used to wrap and unwrap the DEK.
- **Ciphertext:** Encrypted data produced by applying a cipher to plaintext. In terms of the DEK+ format, the wrapped DEK is considered ciphertext.
- **Cleartext:** Unencrypted data that is directly usable without requiring a decryption operation. In terms of this design, a cleartext DEK refers to a DEK that is provided directly through configuration or stored as an unencrypted binary blob.
- **Data file:** An HDF5 file that has been processed using the Encryption VFD.
- **Data Encryption Key (DEK):** Symmetric key used to encrypt and decrypt HDF5 data using the Encryption VFD.
- **Decrypt:** Generic cryptographic term for the inverse operations used to recover ciphertext into plaintext.
- **DEK+ Object:** The on-disk representation of the DEK+ format, Stored in the second page of an encrypted HDF5 file. Future support may allow for the creation of a separate, external file containing the DEK+ Object. Pronounced “DEK plus”.
- **DEK+ format:** Proposed in-memory symmetric key storage format containing metadata, Key Derivation Function parameters, wrapped symmetric DEK, and authentication tag. Used to reference the on-disk format (but not the actual file). Pronounced “DEK plus”.
- **Deserialize:** The process of reconstructing in-memory data structures from their serialized on-disk representation.
- **Encrypt:** Generic cryptographic term for the operations used to transform plaintext into ciphertext.
- **Encrypt-then-MAC:** A cryptographic construction in which data is first encrypted and then authenticated by computing a message authentication code over the resulting ciphertext and selected metadata. During recovery, authentication is verified before any decryption operation is performed.
- **Encryption VFD:** The HDF5 Virtual File Driver responsible for encrypting and decrypting HDF5 data using a recovered DEK. In this design, the Encryption VFD consumes key material but is not responsible for general-purpose key migration or KEK rotation.
- **HMAC:** A keyed message authentication code constructed from a cryptographic hash function. In the DEK+ format, HMAC-SHA256 is used to authenticate the header, salt, IV, and wrapped DEK.
- **Integrity:** Assurance that the authenticated portions of the DEK+ format have not been modified since the authentication tag was generated.
- **Initialization Vector (IV):** A non-secret value used together with a symmetric encryption algorithm to ensure that identical plaintext encrypted under the same key produces different

ciphertext. The IV is necessary for KEK recovery and is stored as part of the DEK+ format. A new random IV should be generated each time the DEK is wrapped using AES-256-CBC.

- **Key Derivation Function (KDF):** A cryptographic algorithm that derives one or more secret keys from a password and associated parameters.
- **Key Encryption Key (KEK):** Key derived from the user password and used exclusively to encrypt/decrypt the DEK.
- **libgcrypt:** A general-purpose cryptographic library developed as part of the GnuPG project. Within the Encryption VFD, `libgcrypt` provides the cryptographic primitives required for random-number generation, symmetric encryption, message authentication, password-based key derivation, and secure memory management. This initial implementation relies on `libgcrypt` for AES-256-CBC, HMAC-SHA256, `scrypt`, cryptographically secure random-number generation, and allocation of sensitive cryptographic material within a protected secure memory pool.
- **Nonce:** A value that is intended to be used only once during a cryptographic operation. Depending on the underlying cryptographic algorithm, a nonce often serves a similar role to an IV.
- **Ordinary Application Memory:** Memory allocated using conventional process allocation mechanisms (for example, `malloc()`) rather than a `libgcrypt` secure memory pool. Ordinary application memory should not be used for long-term storage of secret cryptographic material.
- **Plaintext:** Data before encryption or after a successful decryption. In this design, the plaintext DEK refers to the unwrapped 32-byte binary DEK before it is encrypted into the wrapped DEK.
- **Recover / Recovery:** The complete process of making the plaintext DEK available for use. When using a DEK+ object, recovery consists of reading the DEK+ metadata into ordinary process memory, deriving the KEK material from the user password, verifying the authentication tag, decrypting the wrapped DEK, allocating space within the `libgcrypt` secure memory pool, copying the recovered plaintext DEK into the `libgcrypt` secure memory pool, and securely erasing any temporary cryptographic material.
- **Salt:** A randomly generated, non-secret value combined with the user-provided password during Key Derivation Function (KDF) execution. The salt ensures that identical passwords produce different derived KEKs and reduces the effectiveness of precomputed attacks such as utilizing a rainbow table. The salt is necessary for recovering the KEK and will be stored as part of the DEK+ format.
- **scrypt:** A Key Derivation Function (KDF) designed to securely derive cryptographic keys from passwords or passphrases. Utilizes work factors to deter against leveraging at-rest attack vectors.
- **Secret Material:** Cryptographic information whose disclosure would compromise the security of the DEK+ object or the protected data. In this design, secret material includes the user password, derived KEK material, and the recovered plaintext DEK. Secret material should be

allocated from the `libgcrypt` secure memory pool whenever practical and securely erased when no longer required.

- **Secure Erase:** The process of internally overwriting sensitive memory using a method that prevents compiler optimization from removing the overwrite operation.
- **Secure Memory Pool:** A `libgcrypt`-managed region of locked memory used for storing sensitive cryptographic material. Memory allocated from the secure memory pool is protected against paging to disk and includes additional protections instead to reduce the likelihood of memory disclosure.
- **Serialize:** The process of converting in-memory data structures into their on-disk binary representations.
- **Serialization Format:** The byte-level representation of the DEK+ header and associated fields as written to persistent storage.
- **Structural Validation:** Verification that a DEK+ object is well formed, supported, and internally consistent without requiring the user password or authentication key.
- **Unwrap:** The decryption operation applied to the wrapped DEK to recover the plaintext 32-byte binary DEK.
- **User Password:** The secret entered by the user to derive the KEK.
- **Wrapped DEK:** The at-rest encrypted form of the DEK stored in the proposed DEK+ object format.
- **Wrapping Key:** The first 32 bytes of KEK material derived by the Key Derivation Function. Used exclusively by AES-256-CBC to wrap and unwrap the DEK.
- **Work Factor:** A collection of parameters that determine the computational and memory cost required to execute a Key Derivation Function. Increasing the work factor increases the cost of both legitimate key recovery and password-guessing attacks.

Acknowledgement

This work is supported by the U.S. Department of Energy, Office of Science under award number DE-SC0024823 for Phase I and Phase II SBIR project “Protecting the confidentiality and integrity of data stored in HDF5”

References

- 1) Carper, Clay (2026). Encryption Key Storage Access Mechanisms. Lifeboat LCC
- 2) Tadayoshi Kohno, Niels Ferguson, & Schneier, B. (2010). Cryptography engineering: design principles and practical applications. Wiley Pub., Inc.
- 3) Barker, E. (2020, May 4). Recommendation for Key Management: Part 1 – General. Csrc.nist.gov. <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>

4) Schneier, B. (2015). Applied Cryptography, Second Edition Protocols, Algorithms, and Source Code in C. Indianapolis, Indiana John Wiley & Sons, Inc.

5) OASIS. (2023). “PKCS #11 Specification Version 3.1.” Oasis-Open.org, docs.oasis-open.org/pkcs11/pkcs11-spec/v3.1/pkcs11-spec-v3.1.html. Accessed 1 June 2026.

Revision History

July 9, 2026:	Version 1 circulated for comment within Lifeboat, LLC.